

Kingdom of Saudi Arabia
Ministry of Education
King Khalid University
College of Computer Science



المملكة العربية السعودية
وزارة التعليم
جامعة الملك خالد
كلية علوم الحاسب الآلي

Industrial Control System (ICS) Anomaly Detection

By:

Rand Mohammed
Lama Khalid
Ghada Abdullah

Supervised by

Dr. Amani Mohammad

Submitted to King Khalid University in fulfillment of the requirements for the
B.Sc. Degree in Information Systems

Academic Year 2024-25/1445-46 (Semester II)
Department of Information Systems
College of Computer Science
King Khalid University

Comments by Supervisor(s) / Examiner(s)

Name signature of Supervisor and examiners

Name	Comments	Signature

Abstract

With the widespread digitalization of our lives and the growing connection of devices to the internet, cybersecurity has become a leading research focus in computer engineering. The nature of the internet has certainly led to an increase in both the frequency of cyberattacks and the diversity of their methods.

Industrial control systems (ICS) are essential to industries such as manufacturing, energy, and transportation, forming the backbone of these sectors. However, as they integrate into more connected networks, they face significant cybersecurity risks. Today, the Controller Area Network (CAN) in ICS works reasonably well to provide robust and real-time communication between devices; however, it is not inherently secure and is vulnerable to many attacks, such as Denial of Service (DoS), spoofing, and data injection attacks. This project aims to mitigate these risks by developing an anomaly detection system for CAN-based ICS networks.

ICS can range from small modular controllers mounted on panels to large-scale distributed control networks with millions of connections.

Our project provides an effective solution by using machine learning and utilizing a dataset containing normal and malicious CAN traffic to test ICS cyber-physical security. This project aims to identify attacks, including DoS, spoofing, fuzzy, replay, and data injection attacks.

Table of Content

Comments by Supervisor(s) / Examiner(s).....	2
Abstract.....	3
Table of Content	4
List of Figures	7
List of Tables	9
Chapter 1: Introduction and Background.....	10
1.1 Goal	10
1.2 Motivation	11
1.3 Signification	11
1.4 Method	12
1.5 Report Outline	12
Chapter 2: Literature Review.....	14
2.1 Overview of Industrial Control Systems (ICS)	14
2.1.1 Key Components of ICS	15
2.1.2 Threat Landscape for ICS	16
2.2 CAN Protocol Overview	17
2.3 The Controller Area Network (CAN) and its vulnerabilities	18
2.3.1 Vulnerabilities of CAN in Industrial Control Systems.....	18
2.4 Machine Learning for Anomaly Detection in ICS	19
2.5 Problem Statement	22
2.6 Research Questions	23
2.7 Existing Systems	24
2.7.1 Comparison Table.....	27
Chapter 3: Project Management.....	29
3.1 Approach	29
3.1.1 Agile Software Development Life Cycle	29
3.1.2 Process Model Justification	30
3.2 Risk Management.....	31
3.2.1 Identified Risks.....	31

3.2.2 Risk Management Plan	32
3.3 Project Plan	32
Chapter 4: Requirments and Analysis.....	34
4.1 System Requirements	34
4.1.1 Functional Requirements	34
4.1.2 Non-Functional Requirements.....	35
4.2 System Scenario	36
4.3 System Models	37
4.3.1 Use Case Diagram	37
4.3.2 Sequence Diagram	38
4.3.3 Activity Diagram	38
Chapter 5: System Design.....	40
5.1 Product Features	40
5.2 Project Implementation	43
5.3 High Level Design	44
5.3.1 Dataflow Diagram	44
Chapter 6: PROJECT IMPLEMENTATION	46
6.1 Coding	46
6.2 Verification.....	61
6.3 Validation.....	61
Chapter 7: PROJECT TESTING (EVALUATION).....	63
7.1 Metrics.....	63
7.2 Experimental Setup	64
7.3 The Experiments	64
7.3.1 Participants	64
7.3.2 Treatments.....	64
7.3.3 Procedures	65
7.3.4 Data Analysis	65
7.4 Results	65
7.4.1 Isolation Forest	65
7.4.2 Random Forest.....	69
7.4.3 SVM.....	73
7.4.4 Accuracy Comparison	79
7.5 Validity.....	81

7.5.1 Potential Threats to Validity	81
7.6 Summary of Findings (Discussion).....	82
7.6.1 Explanation of Outcomes	82
7.6.2 Implications of Results	83
7.6.3 Solution Review	84
7.6.4 Project Review.....	85
7.6.5 Key Skills	85
7.6.6 Summary of Discussion.....	85
Chapter 8: Conclusion and Future Works.....	87
8.1 Conclusion.....	87
8.2 Limitations	88
8.3 Future Works.....	88
8.4 Summary	89
References.....	90
APPENDICES	94
Appendix A: Project Plan.....	94
Appendix B: User Requirements Specification	98
Appendix C: Software Testing Document	99
Appendix D: Software Validation Document	101
Appendix E: Experimental Data and Results Document	103

List of Figures

Figure 2.1: Key Components of ICS [11]	16
Figure 2.2: CAN protocol Component [15]	18
Figure 2.3: A Machine Learning Approach for Anomaly Detection [6]	22
Figure 2.4: Industrial Control Systems (ICS) in Energy and Utilities [23]	25
Figure 2.5: (SCADA) Systems [21]	26
Figure 2.6: Advanced Metering Infrastructure (AMI) [22]	27
Figure 3.1: Agile Methodology [24]	30
Figure 3.2: Gantt Chart	33
Figure 4.1: Use Case Diagram	37
Figure 4.2: Sequence Diagram	38
Figure 4.3: Activity Diagram	39
Figure 5.1: Code of Upload the dataset	41
Figure 5.2: Code of Data Processing	41
Figure 5.3: Code of training and testing sets	42
Figure 5.4: Code of Train and Evaluate the model	42
Figure 5.5: Screen capture of the Google Colab interface.[25]	43
Figure 5.6: Dataflow Diagram	45
Figure 6.1: Google Colab	47
Figure 6.2: Label Categories	49
Figure 6.3: Load Dataset	50
Figure 6.4: All Numerical Columns	51
Figure 6.5: Feature Selection	52
Figure 6.6: Split Dataset	53
Figure 6.7: Isolation Forest First Multi-class mode	54
Figure 6.8: Isolation Forest Multi-class mode	57
Figure 6.9: Isolation Forest Binary mode	57
Figure 6.10: Random Forest (RF) for Muli-class Mode	59
Figure 6.11: Random Forest (RF) for Binary Mode	59
Figure 6.12: SVM for Multi-class mode	60
Figure 6.13: SVM for Binary mode	61

Figure 7.1: Isolation Forest (IF) accuracy results for multi-class mode	66
Figure 7.2: Isolation Forest (IF) accuracy results for binary mode	67
Figure 7.3: Isolation Forest Confusion Matrix for multi class mode.....	68
Figure 7.4: Isolation Forest Confusion Matrix for binary mode.....	69
Figure 7.5: Random Forest (RF) Accuracy Results for multi class mode	70
Figure 7.6: Random Forest (RF) Accuracy Results for binary mode	70
Figure 7.7: Random Forest Confusion Matrix for multi class mode	72
Figure 7.8: Random Forest Confusion Matrix for binary mode	73
Figure 7.9: SVM for multi class mode.....	74
Figure 7.10: SVM for binary mode.....	76
Figure 7.11: SVM Confusion Matrix for multi class mode	77
Figure 7.12: SVM Confusion Matrix for binary mode	79
Figure 9.1: Gantt Chart	97

List of Tables

Table 2.1: Comparison of ICS, SCADA, AMI, and CAN-based Anomaly Detection Approaches.	28
Table 7.1: Comparison of Models for multi class mode.....	80
Table 7.2: Comparison of Models for Binary mode.....	80
Table 9.1: Milestones& Deliverables	96
Table 9.2: Functional Requirements	98
Table 9.3: Non-Functional Requirements.....	98
Table 9.4: Test Cases Table	100
Table 9.5: Bugs Table	100
Table 9.6: Model Evaluation Table1	102
Table 9.7: Model Evaluation Table2	102
Table 9.8: Models Used	103

Chapter 1: Introduction and Background

Industrial control systems (ICS) are important for most of the critical infrastructure sectors, including energy, manufacturing, transportation, etc. These systems are designed to regulate the actual processes, establish operational efficiency, and mitigate risks. In contrast to corporate IT systems, ICS environments often operate with a lower level of security, increasing the risk significantly, particularly in critical infrastructure areas, where the consequences of a security breach can be severe. This is exemplified by the Stuxnet attack in 2010, which targeted Iran's nuclear centrifuges, and the coordinated cyberattacks on Ukraine's power networks in October 2015 and December 2016. These events underscore the importance of protecting ICS against cyberattacks, as the potential economic impacts, environmental effects, and threats to national security are substantial. Developing a robust protection system for CAN-based ICS networks is essential, as it can significantly reduce the risk of attacks on critical infrastructure, thereby safeguarding lives and property that could be endangered in the event of a CAN attack. This project aims to address these challenges through a machine learning-based anomaly detection system.

1.1 Goal

The main objective is to use a machine learning-based anomaly detection system (ML-based ADS) to improve the security of Controller Area Network-based Industrial Control Systems (CAN-ICT) in ICS. Specifically, the following outcomes are to be achieved:

1. Design and Implement an Anomaly Detection System:
Develop a CAN traffic analysis system in ICS environments to enhance the ability to identify cyber-attacks, such as Denial of Service (DoS), spoofing, fuzzy attacks, and data injection attacks.
2. Leverage Machine Learning Models:
Apply ML algorithms to classify CAN traffic as normal or malicious. The system will be trained using a real-world and synthetic CAN dataset containing various attack scenarios.
3. Contribute to the Field of ICS Cybersecurity:
Provide recommendations on improving cybersecurity in CAN-based ICS

networks and contribute to the development of further research articles.

1.2 Motivation

ICS systems control essential operations in critical infrastructures and organizations, and their breakdown can lead to significant losses and threats to public health. As ICS environments become more connected, their exposure to cyber threats increases, raising the risk of sophisticated attacks. This situation highlights the necessity for industries to adhere to strict regulatory measures on cybersecurity. The proposed solution aims to reduce economic losses caused by cybercriminals and contributes to the advancement of ICS cybersecurity by providing effective measures to secure industrial processes and ensure reliable operations.

1.3 Signification

The project's value lies in enhancing the security of CAN-based ICS, which manage vital processes in industries that are the backbone of the modern world. If these systems are tampered with, attackers can destabilize businesses, incur immeasurable costs, and jeopardize public safety while also harming the environment. Therefore, it is essential to safeguard these systems against such threats through the proposed solutions. Although CAN is widely used in industrial and automotive systems today, it was not originally designed with security in mind. However, by employing machine learning for anomaly detection, this project aims to address dynamic threats and prevent attacks before they are successful.

1.4 Method

The project is carried out in several key steps to develop an effective anomaly detection system for CAN-based Industrial Control Systems (ICS):

1. Problem Identification and Research:

The process begins by evaluating the attacks on the CAN protocol, which is widely used in ICS networks, and identifying the lack of enhanced security features. Specific cyberattacks affecting CAN traffic that will be researched include DoS, spoofing, fuzzy, and data injection attacks.

2. Dataset Selection and Preprocessing:

A selection of normal and malicious CAN traffic will be used, including the ICS-Flow dataset, to assess ICS cyber-physical security monitoring. The first process in the data preprocessing stage involves cleaning the data to eliminate any interference with the ML algorithms.

3. Model Testing and Evaluation:

The trained model will be tested to determine how accurately it can detect cyber-attacks using a test dataset.

4. Results Analysis and Documentation:

The performance analysis of the system will be conducted, with results indicating the effectiveness of the designed model in identifying anomalies with desirable accuracy and low false alarm rates.

1.5 Report Outline

The structure of the documentation is as follows:

- Chapter 1

This chapter provides the background of the project, outlining the objectives, relevance, rationale, and approach used.

- Chapter 2

This chapter synthesizes existing research, highlighting the current state of

knowledge, identifying emerging gaps, and presenting the formulated problem of the project.

➤ Chapter 3

This chapter discusses the project management approach, including risk management strategies and the overall project schedule.

➤ Chapter 4

This chapter covers the requirements analysis, identifying the extent of project requirements through elicitation, system requirements, use personas, and system models.

➤ Chapter 5

This chapter explains all aspects of system design, including product features, user interfaces, databases, and high-level solution design.

➤ Chapter 6

This chapter presents the conclusions drawn from the project, providing an overview of the findings and discussing potential directions for improving the project's results.

➤ Appendix

This section includes additional information that supports the collected data.

Chapter 2: Literature Review

This chapter provides information about the theoretical background and the literature analysis related to the project. It outlines the study's aims and objectives, presenting a rationale for the proposed research methods used in this project.

2.1 Overview of Industrial Control Systems (ICS)

Industrial control systems (ICS) serve as network resource for managing operations across various sectors of the economy, including energy, transportation, and manufacturing. Generally, all ICS prioritize the continuity and reliability of operations; however, security concerns often take a back seat as a result [5]. In an ever-evolving cyberspace, ensuring ICS cybersecurity has become paramount. For instance, the deterministic nature of the Controller Area Network (CAN) makes systems using CAN more vulnerable, as they lack intrinsic security features. The potential consequences of these vulnerabilities include operational disruptions, loss of life, and abnormal physical developments. The Controller Area Network (CAN) protocol was initially designed in the 1980s for automotive systems, focusing on efficiency, simplicity, and reliability within isolated environments, rather than on cybersecurity [1]. At that time, vehicles and other industrial systems were not connected to external networks, so security was not a primary concern. The lack of built-in security in CAN can be attributed to several key factors:

1. **Original Design Philosophy:** CAN was initially developed for cars and other enclosed, highly controlled systems, operating in environments where all devices were known and did not require encrypted communication. This assumption is no longer valid, as modern CAN-based systems are vulnerable to external connectivity [3].
2. **No Authentication:** A recent survey of CAN revealed that, although CAN uses a broadcast communication model (where every message is sent to all nodes), there is no security system in place to verify the legitimacy of message's source. This lack of authentication allows any connected node to send messages, making the system susceptible to spoofing attacks [3].

3. **No Encryption:** Unlike other automotive communication standards, messages exchanged through CAN are not encrypted, making them easy to intercept or modify. Attackers can eavesdrop on network messages and even alter them without any protective measures in place [2].
4. **Lack of Access Control:** CAN does not distinguish between critical and non-critical messages or between trusted and untrusted devices [3]. Any node on the network can send commands, including potentially malicious ones, leading to unauthorized control over critical systems.
5. **Limited Adaptability for Security Enhancements:** While real-time communication is crucial, the absence of built-in security restricts the implementation of secure protocols to meet new security requirements, such as authentication and data encryption.

2.1.1 Key Components of ICS

ICS setups consist of several critical elements that control and monitor physical processes, as illustrated in Figure 2.1. These components include:

- **Sensors:** Devices that measure parameters such as temperature and pressure at intervals close enough to be considered instantaneous.
- **Actuators:** Mechanisms that use control signals to alter physical functions, such as motors and valves.
- **Human-Machine Interface (HMI):** Interfaces that provide operators with tools for monitoring and controlling systems.
- **Control Servers and Remote Terminal Unit (RTUs):** Systems that share data among components, facilitating communication between sensors and other elements within the process.
- **Communication Networks:** Specialized data links that connect sensors, process control systems (PCS), and actuators. Unlike conventional information networks, these networks are designed for high speed and real-time performance.

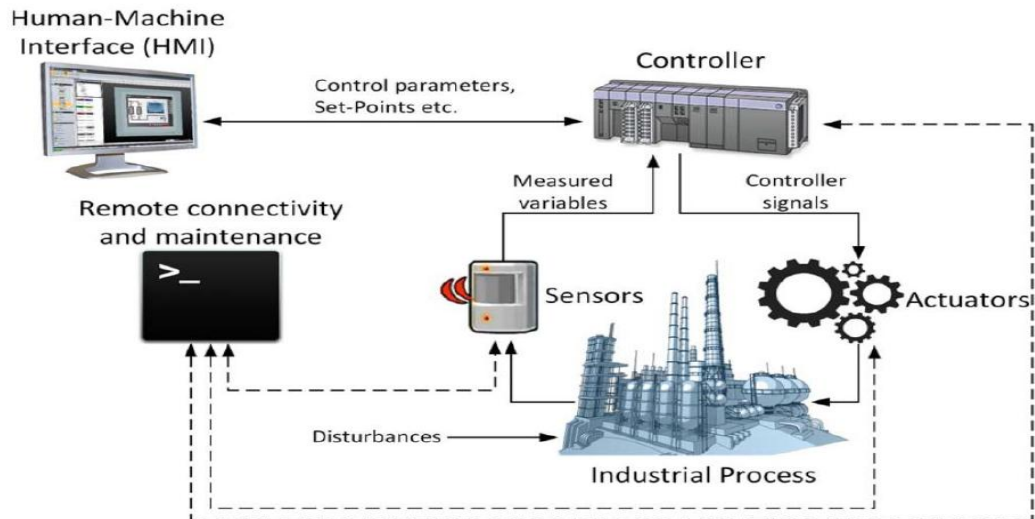


Figure 2.1: Key Components of ICS [11]

2.1.2 Threat Landscape for ICS

ICS systems have been integrated with corporate networks and the internet, significantly raising their vulnerability to cyberattacks. The transition to Industry 4.0 and the Internet of Things has introduced new challenges, particularly as traditional ICS environments were often 'air-gapped' for security [10]. Notable threats include:

- **Denial of Service (DoS) Attack:** This attack involves overwhelming the ICS network with excessive traffic or requests beyond its capacity. Such flooding can lead to intolerable delays in communication between controllers and devices, resulting in longer response times and potential system degradation or failure of essential operations.
- **Data Injection Attack:** This attack involves tampering with the transmission media to mislead control systems and operators. By compromising the normal flow of communication, attackers can provide the ICS with false information. For example, if a temperature sensor transmits incorrect data, it may trigger erroneous actions, such as shutting down equipment or altering critical processes. This interference can disrupt operations and potentially lead to equipment failure or unsafe conditions.
- **Spoofing attack:** In this scenario, an adversary impersonates a legitimate device or system component within the ICS, gaining unauthorized control over processes. By mimicking a valid device, an attacker can send false commands

or data, influencing industrial operations. This can lead to unauthorized changes in system behavior, such as alterations in the production line or disabling safety systems. Spoofing undermines trust within the system, as operators may believe they are receiving accurate information from legitimate devices when, in fact, they have been infiltrated.

- Fuzzy attack: This type of attack involves injecting incorrect or illogical data to exploit protocol flaws within the ICS. Fuzzy attacks can overwhelm or confuse the system's control logic, leading to unpredictable or faulty behavior. Attackers craft specific malicious inputs to exploit vulnerabilities in communication protocols, such as the Controller Area Network (CAN).

2.2 CAN Protocol Overview

The CAN protocol allows for high-rate data transfer between devices on a local network bus. Its key features include [15]:

- Priority-Based Arbitration: More critical messages can be prioritized, allowing them to be transmitted ahead of less urgent messages.
- Broadcast Communication: All messages sent are received by every node, facilitating easier delivery.

However, fundamental security measures that CAN lacks should be highlighted to minimize unauthorized access and attacks. In a CAN system, data communication occurs between multiple CAN nodes, which consist of three primary components, as shown in Figure 2.2:

1. Host Processor: Each node (e.g., CAN Node X, CAN Node Y, and CAN Node Z) contains a host processor responsible for determining what data to send and how to interpret received data.
2. CAN Controller: Data is initiated by the host processor and sent to the CAN controller, which prepares the data in the form of CAN frames and manages the sending and receiving of data on the CAN network.

3. CAN Transceiver: The CAN controller sends CAN frames to the transceiver, which acts as both a transmitter and receiver. It converts the data from the controller into electrical signals that can be transmitted over the bus lines.

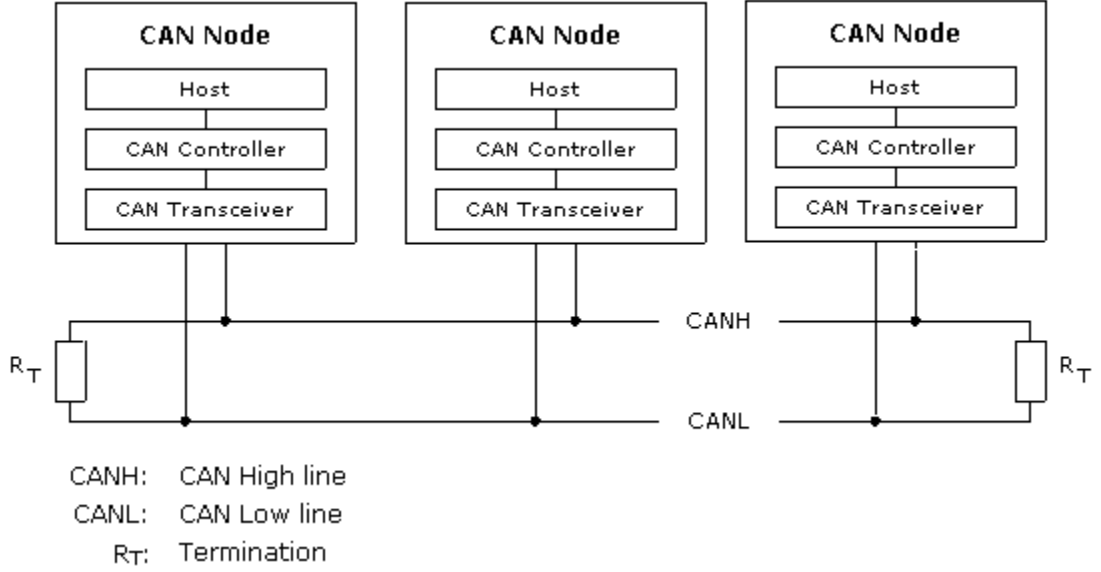


Figure 2.2: CAN protocol Component [15]

2.3 The Controller Area Network (CAN) and its vulnerabilities

Originally developed for automotive systems, the CAN protocol is now widely used in ICS due to its efficiency in facilitating real-time device communication. However, it is not built with adequate security features such as encryption and authentication; hence, it is an open door for attacks such as DoS, spoofing, and data injection attacks. Studies in [1] and [2] indicate that these vulnerabilities render CAN a weak link in ICS, necessitating enhanced security measures. The goal of this project is to design a machine learning-based anomaly detection system to mitigate such security threats. In the following subsections, we will describe the CAN protocol and discuss its vulnerabilities.

2.3.1 Vulnerabilities of CAN in Industrial Control Systems

In ICS environments, the widespread use of the CAN protocol introduces several significant vulnerabilities [11], [12]:

- **Absence of Authentication:** CAN lacks the capability to verify the identity of connected nodes, allowing attackers to clone legitimate devices and broadcast fraudulent messages.
- **No Data Encryption:** CAN messages are transmitted without encryption, making the network susceptible to impersonation and eavesdropping attacks.
- **Broadcast Nature:** The broadcast feature of CAN means that any malicious message injected into the network is received by all devices. Without a protocol for authenticating or verifying messages, devices cannot distinguish between legitimate and malicious communications, allowing attackers to feed incorrect information into the system, potentially leading to failures or unauthorized control over processes.
- **No Message Authentication Codes (MACs):** Messages exchanged between nodes cannot be checked for integrity, enabling attackers to alter them without detection.
- **Arbitration Exploits:** The priority-based arbitration mechanism in CAN is also vulnerable. An attacker can disrupt normal operations by continuously sending high-priority messages, preventing legitimate lower-priority messages from being transmitted.

2.4 Machine Learning for Anomaly Detection in ICS

Machine learning is applied to analyse large amounts of data on ICS networks and investigate deviations within the network. Among the most effective algorithms are decision trees, Support Vector Machines (SVM), and the increasingly popular neural networks. Studies cited in [6] demonstrate that ML techniques are effective in identifying potential risks in CAN-based systems, aligning with this project's goal of distinguishing between normal and anomalous network traffic. The process for anomaly detection in Industrial Control Systems (ICS) focuses on four key areas: Data Collection, Data Engineering, Model Training, and Classification, as illustrated in Figure 2.3.

1. Data Collection

- Objective: Gather relevant data from various sources to create a comprehensive dataset for analysis.
- Sources: Collect information from instruments measuring operational parameters such as temperature, pressure, and flow rate.
- Control Systems: Acquire data from programmable logic controllers (PLCs).
- Historical Data: Include previous logs of operational data, such as logs containing standard or expected behaviours.
- Data Types:
 - Time-Series Data: Segments of data collected over time, essential for monitoring processes.
 - Event Data: Sudden occurrences, such as alarm events and system statuses.

2. Data Engineering

- Objective: Process and transform the collected data into a format suitable for analysis.
- Key Processes:
 - Data Cleaning: Filter necessary data and remove noise and duplicates to ensure high-quality input.
 - Feature Engineering: Identify and select important features for anomaly detection, which may involve:
 - Aggregation: Temporal aggregation of data.
 - Normalization: Standardizing features to ensure consistent ranges.
 - Transformation: Modifying data to improve model performance.
 - Data Splitting: Divide the dataset into training, cross-validation, and testing sets to evaluate model performance.

3. Model Training

- Objective: Develop a machine learning model to predict anomalies based on the processed data.
- Key Processes:
 - Algorithm Selection: Choose appropriate machine learning algorithms based on dataset features and the problem at hand.
 - Training Phase: Familiarize the model with the training subset to recognize deviations.
 - Model Fitting: Fine-tune model parameters to minimize error estimates through optimization techniques (e.g., gradient descent).
 - Hyperparameter Tuning: Use cross-validation to adjust performance characteristics, such as 'learning rate' and other parameters.

4. Classification

- Objective: Categorize incoming data as either normal or anomalous.
- Key Processes:
 - Real-Time Inference: Apply the developed model to process incoming real-time data in ICS.
 - Anomaly Scoring: Assign an anomaly score to each incoming data point by comparing it to learned patterns.
 - Thresholding: Establish criteria for distinguishing typical data points from those in the abnormal range.
 - Decision Making: Based on classification results, the system can alert operators or automatically respond to intrusions, enhancing overall security.

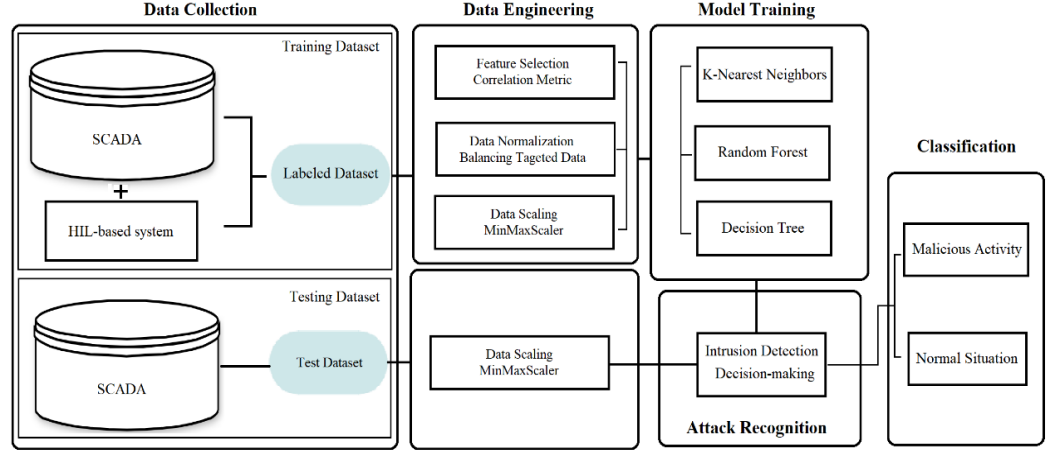


Figure 2.3: A Machine Learning Approach for Anomaly Detection [6]

2.5 Problem Statement

New technologies have emerged in the ICS field, particularly CAN-based technologies, which unfortunately lack critical security features. As a result, current protective measures often fail to identify complex cyberthreats in real time. This project aims to address this issue by developing an anomaly detection system that leverages various machine learning algorithms to detect multiple types of cyberattacks, including DoS, spoofing, and fuzzy attacks, as they occur.

From a mathematical perspective, the proposed anomaly detection framework can be outlined as follows [5], [6]:

1. **Data Framework:** Let $X \in R^n$ denote the feature vector obtained from the incoming data within the ICS, where n is the number of features extracted from sensor outputs, control signals, and other pertinent information.
2. **Modeling Standard Operations:** Construct M_N , a model for normal operational behavior utilizing a training dataset D_N , which consists of examples representing standard system functions:

$$M_N = f(X; \theta_N)$$

Here, f refers to a machine learning algorithm trained by θ_N to identify expected behavior and detect anomalies. For any incoming data point x , calculate its anomaly score $S(x)$ based on its divergence from the established normal model:

$$S(x) = \text{distance}(x, M_N)$$

Depending on the dataset properties, this distance metric can employ various methods, including Euclidean or Mahalanobis distance.

3. Setting Classification Criteria: Establish a threshold T for the anomaly score; if $S(x) > T$, the data point will be flagged as anomalous:

If $S(x) > T$, then x is deemed anomalous

4. Implementation of Response Protocols: The system must include a responsive mechanism that activates whenever it detects an anomaly or potential threat. This mechanism may initiate an action or event deemed safe to counter the perceived threats.

This mathematical framework can significantly enhance the protection of critical ICS infrastructures against complex cyber threats in today's digital landscape.

2.6 Research Questions

The following research questions guide the study:

- How effectively do machine learning models detect cyberattacks on CAN-based ICS, such as DoS, data injection, spoofing, and fuzzy attacks?
- What challenges may arise when applying anomaly detection in ICS networks?
- Can existing machine learning algorithms be scaled to effectively detect anomalies in ICS networks of varying sizes?

2.7 Existing Systems

The growing number of industrial control systems (ICS) in Saudi Arabia has led to significant advancements in cybersecurity measures to protect critical infrastructure. Saudi Aramco has been at the forefront of deploying ML technologies to detect cyber threats and safeguard the company from attacks such as DoS and data injection attacks. Additionally, the National Cybersecurity Authority (NCA) coordinates national efforts to enhance the protection of vital networks and systems, leveraging AI and ML for rapid threat identification. These existing systems reflect Saudi Arabia's increasing focus on strengthening cyber-physical security within industrial sectors.

1. Security in Electric Utility Control Systems [19]

The oil and gas, along with the electricity sector in Saudi Arabia, heavily depend on ICS. These systems are crucial for the operation of extensive infrastructures such as oil refineries, power processing plants, and water treatment facilities.

➤ Features

- Automated monitoring and control
- Real-time data collection
- Preventing cyber threats using machine learning and advanced analytical techniques.

➤ Limitations

- Some ICS systems rely on outdated technology, which may not integrate well with current security solutions.
- These systems are often targeted by complex attack strategies such as data injection and denial-of-service (DoS).
- Updating or patching ICS while operational is challenging, leaving many security vulnerabilities unaddressed.



Figure 2.4: Industrial Control Systems (ICS) in Energy and Utilities [23]

2. SCADA Security in the Light of Cyber-Warfar [17]

SCADA systems are widely implemented in Saudi industries to manage various infrastructural facilities, including energy, water, and transport sectors.

➤ Features

- Real-time data collection at the sensor level to maximize efficiency and identify faults.
- Alerts broadcasted in case of system malfunctions for immediate response.
- Use of Intrusion Detection Systems (IDS) and firewalls to protect against external threats.

➤ Limitations

- Remote access capabilities make SCADA systems vulnerable to cyber threats, particularly from external actors.
- Integrating new features into century-old SCADA systems can disrupt business continuity while improving compliance with current IT security standards.
- Vulnerability to insider threats, where authorized personnel may misuse their access.

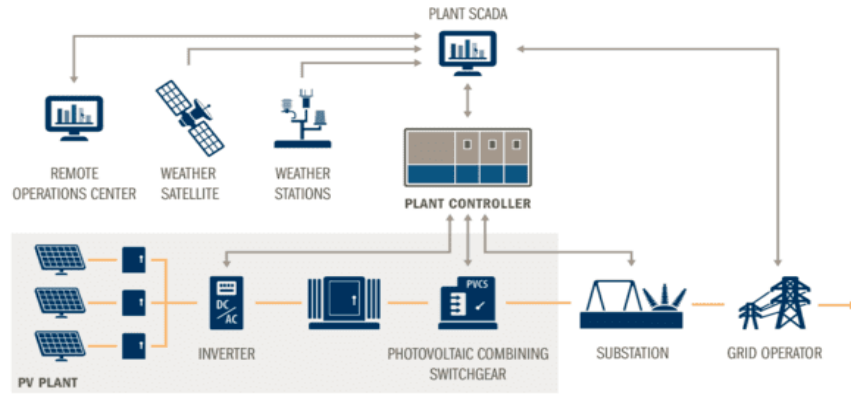


Figure 2.5: (SCADA) Systems [21]

3. Digital Certificate Management in Advanced Metering Infrastructure Networks [20]

The Saudi government has adopted Advanced Metering Infrastructure (AMI) systems to optimize energy control and utilize smart meters for electricity billing.

➤ Features

- AMI enhances the accuracy of meter readings, improving the efficiency of electricity, gas, and water usage.
- Consumers gain access to real-time usage data, helping them monitor their energy consumption.
- AMI enables consumer interaction with utilities for outages management and demand response.
- Security features protect consumer information through robust code and communication interfaces.

➤ Limitations

- The large number of connected devices (smart meters) increases the risk of cyberattacks, such as data breaches and tampering.
- The collection of real-time consumer data raises privacy concerns and potential data misuse.

Advanced Metering Infrastructure (AMI)

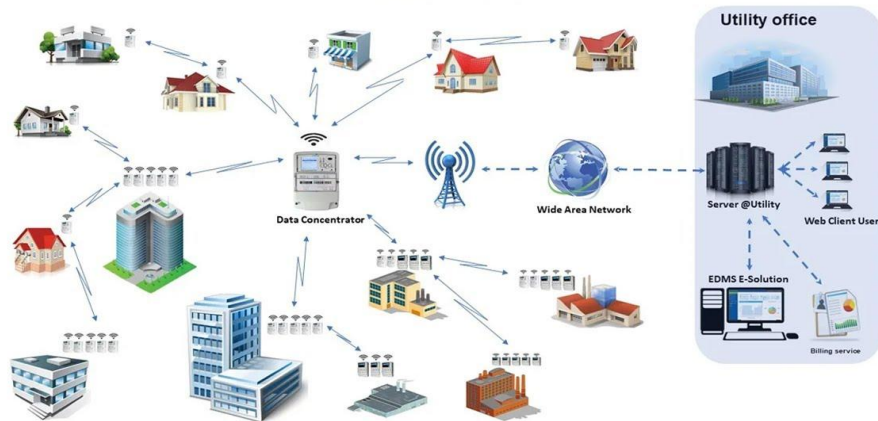


Figure 2.6: Advanced Metering Infrastructure (AMI) [22]

2.7.1 Comparison Table

Criteria	Security in Electric Utility Control Systems [19]	SCADA Security in the Light of Cyber-Warfar [17]	Digital Certificate Management in Advanced Metering Infrastructure Networks [20]	Industrial Control System (ICS) Anomaly Detection
Focus	Cybersecurity Threats and Resilience	Security Protocols and Attack Prevention	Data Privacy and Cybersecurity	Machine Learning Applications in Cybersecurity
Approach	Rule-based control, PID control algorithms	Isolation Forest, SVM, K-Means Clustering, Regression Analysis, Time Series Analysis	Decision Trees, Z-Score Analysis, Dynamic Thresholding, Data Encryption (AES, RSA)	Supervised (e.g., Decision trees, SVM) and Unsupervised (e.g., K-Means Clustering)

Dataset Used and Size	SWaT Dataset (approximately 2.1 million data points)	Gas Pipeline SCADA Dataset (approximately 1 million data points)	Smart Home Dataset (approximately 10,000 to several million rows of data points)	ICS-Flow Dataset (It encompasses over 25 million raw network packets, network flow records, and process variable logs.)
Programming Language	IEC 61131-3, ladder logic	Industrial languages (often C/C++), SCADA protocols	Often C/C++ for embedded systems	Python
Attack Types Addressed	Malware, phishing, DoS, zero-day attacks	Network-based attacks: DoS, MitM, protocol exploits, tampering attacks	Data manipulation, unauthorized access, replay attacks	CAN-specific attacks: DoS, spoofing, fuzzy, replay, data injection attacks
Detection Speed	Moderate, affected by system and process complexity	Variable, slower in remote systems	Limited by device processing power	Real-time using Google Colab for interactive execution and analysis

Table 2.1: Comparison of ICS, SCADA, AML, and CAN-based Anomaly Detection Approaches.

Chapter 3: Project Management

Effective project management is crucial for the successful development of the anomaly detection system tailored for CAN-based ICS. This chapter outlines the implementation strategies of the project management plan, detailing the systematic direction of the project, the realization of planned goals and objectives, and the mitigation of risk factors. It involves selecting the most appropriate approach to project development and identifying potential risks. The working procedures and organizational methods are described in the context of ensuring smooth project progression and timely completion. To align with best practices in software development and cybersecurity studies, the planning process was designed to adapt to the specific requirements of the project.

3.1 Approach

The framework employs a structured and iterative approach to software development. Given the complexity of building a machine learning-based anomaly detection system for ICS networks, the primary model chosen is the Agile Software Development Life Cycle (ASDLC). Agile methodologies facilitate adaptability, allowing for the incorporation of new ideas, technical requirements, or data analysis insights into the project without compromising time constraints. This flexible approach ensures that the development process remains responsive to evolving needs and challenges.

3.1.1 Agile Software Development Life Cycle

ASDLC consists of short development cycles, known as sprints, typically lasting between 2 to 4 weeks. Each sprint, as illustrated in Figure 3.1, involves a working cycle that includes planning, design, development, testing, deployment, and review of a specific part of the system. This structure allows for the rapid incorporation of improvements and feedback, making Agile significantly more responsive than traditional, structured development systems.

The Agile approach aligns well with project requirements, as it mirrors the iterative nature and cycle times necessary for training machine learning models [24]. This flexibility ensures that the project can adapt to changes and continuously enhance its outputs throughout the development process.



Figure 3.1: Agile Methodology [24]

Several key features of the Agile methodology influenced its selection for this project:

- Collaboration: Frequent communication with stakeholders allows for adjustments to the project's goals and ensures alignment with industry standards.
- Real-Time Feedback: Models can be tested and receive immediate feedback, enabling timely responses and adaptation to changes.

3.1.2 Process Model Justification

Several factors influenced the decision to implement the ASDLC model for this project [24]:

- Complexity of Machine Learning Development: Machine learning programs require constant experimentation, evaluation, and retraining to

enhance reliability and accuracy. Agile's experimental and iterative design makes it particularly well-suited for this type of work.

- **Evolving Nature of ICS Cybersecurity:** The landscape of threats facing ICS cybersecurity is constantly changing, with new vulnerabilities identified regularly. Agile's flexibility allows for continuous adjustments and improvements to the anomaly detection system, ensuring it remains effective against emerging threats.

3.2 Risk Management

Risk must be addressed in any project, especially when developing a Machine Learning-based cybersecurity tool for an ICS infrastructure. This section identifies potential risks that could hinder the project's success and outlines strategies for managing those risks.

3.2.1 Identified Risks

1. Data Availability and Quality:

- **Risk:** Implementing an anomaly detection system may be challenging if suitable datasets containing both benign and malicious CAN traffic data are lacking, or if the available dataset is of low quality.
- **Mitigation:** Utilize publicly available datasets and generate synthetic data as needed to ensure robust training of the anomaly detection system.

2. Cybersecurity Threat Evolution:

- **Risk:** New attacks on ICS networks could emerge during the project, potentially undermining the effectiveness of the anomaly detection system.
- **Mitigation:** Conduct periodic updates on the state of the art in ICS cybersecurity threats, as reported in academic and grey literature. The anomaly detection system will incorporate mechanisms for updates and adaptability to ensure it remains relevant as new threats arise.

3.2.2 Risk Management Plan

An effective risk management strategy will be established to facilitate risk identification, control, and evaluation, allowing for stakeholder engagement at various levels throughout the project implementation [8]. The strategy will consist of:

- Risk Identification and Evaluation: Conduct semi-annual or annual assessments to uncover new risks and reassess previously identified risks.
- Risk Response Strategy: Develop an action plan to implement when a risk materializes, along with contingency plans for areas with high-risk potential.
- Risk Monitoring: Regularly review and report on risks using risk logs and a variety of project management tools to assess the current risk status within the organization.

3.3 Project Plan

The Gantt chart presented in Figure 3.2 illustrates the project's timeline and key milestones. This chart helps track progress and ensures that all tasks are completed on schedule.

		Name	Duration	Start	Finish	Resource Names
1	👤	📁 Preliminary Research and Project Concept	70 days?	8/29/24 8:00 AM	12/4/24 5:00 PM	Rand Mohamed;Lama Khalid;Ghada Abdullah
2	📅	Make Primary Research	3 days?	8/29/24 8:00 AM	9/2/24 5:00 PM	Rand Mohamed
3	📅	Project Proposal Submission (Using Google Form)	3 days?	8/31/24 8:00 AM	9/4/24 5:00 PM	Lama Khalid
4	📅	Project Plan	4 days?	9/5/24 8:00 AM	9/10/24 5:00 PM	Ghada Abdullah
5	📅	Gantt Chart	0.857 days?	9/11/24 8:00 AM	9/11/24 3:51 PM	Ghada Abdullah;Lama Khalid;Rand Mohamed
6	👤	📁 Chapter 1 : Introductation and Background	59 days?	9/13/24 8:00 AM	12/4/24 5:00 PM	Rand Mohamed;Lama Khalid;Ghada Abdullah
7	📅	Goal	1.125 days?	9/13/24 8:00 AM	9/16/24 9:00 AM	Rand Mohamed
8	📅	Motivation	2 days?	9/16/24 8:00 AM	9/17/24 5:00 PM	Lama Khalid
9	📅	Method	1.75 days?	9/18/24 8:00 AM	9/19/24 3:00 PM	Ghada Abdullah
10	📅	Report Outline	2 days?	9/19/24 8:00 AM	9/20/24 5:00 PM	Rand Mohamed;Lama Khalid;Ghada Abdullah
11	👤	📁 Chapter 2 : Literature Review & chapter 3 : Project Managment	54 days?	9/20/24 8:00 AM	12/4/24 5:00 PM	Rand Mohamed;Lama Khalid;Ghada Abdullah
12	📅	Problem Statement	2 days?	9/20/24 8:00 AM	9/23/24 5:00 PM	Rand Mohamed
13	📅	Research Questions	3 days?	9/24/24 8:00 AM	9/26/24 5:00 PM	Rand Mohamed
14	📅	Approach	3 days?	9/27/24 8:00 AM	10/1/24 5:00 PM	Lama Khalid
15	📅	Risk Management	2 days?	10/1/24 8:00 AM	10/2/24 5:00 PM	Lama Khalid
16	📅	Project Plan	12.125 days?	10/3/24 8:00 AM	10/21/24 9:00 AM	Ghada Abdullah
17	📅	Mid Exam Presentation	4 days?	10/19/24 8:00 AM	10/24/24 5:00 PM	Ghada Abdullah
18	👤	📁 Chapter 4 : Requirements and Analysis	4 days?	10/25/24 8:00 AM	10/30/24 5:00 PM	Rand Mohamed;Lama Khalid;Ghada Abdullah
19	📅	Requirement Eliotatation	2 days?	10/25/24 8:00 AM	10/28/24 5:00 PM	Rand Mohamed
20	📅	System Requirements	2 days?	10/28/24 8:00 AM	10/29/24 5:00 PM	Lama Khalid
21	📅	Personas	2 days?	10/29/24 8:00 AM	10/30/24 5:00 PM	Ghada Abdullah
22	📅	System Models	0.667 days?	10/30/24 8:00 AM	10/30/24 2:20 PM	Ghada Abdullah;Rand Mohamed;Lama Khalid
23	👤	📁 Chapter 5 : System Design	5 days?	11/1/24 8:00 AM	11/7/24 5:00 PM	Rand Mohamed;Lama Khalid;Ghada Abdullah
24	📅	Product Features	1 day?	11/1/24 8:00 AM	11/1/24 5:00 PM	Rand Mohamed
25	📅	User Interface	2 days?	11/4/24 8:00 AM	11/5/24 5:00 PM	Lama Khalid
26	📅	Data Storage	2 days?	11/5/24 8:00 AM	11/6/24 5:00 PM	Ghada Abdullah
27	📅	High Level Design	2 days?	11/6/24 8:00 AM	11/7/24 5:00 PM	Rand Mohamed;Lama Khalid;Ghada Abdullah
28	👤	📁 Chapter 6 : Conclusions and Future Work	38 days?	10/14/24 8:00 AM	12/4/24 5:00 PM	Rand Mohamed;Lama Khalid;Ghada Abdullah
29	📅	Future Work	35 days?	10/14/24 8:00 AM	11/29/24 5:00 PM	Rand Mohamed;Lama Khalid;Ghada Abdullah
30	📅	Summary	0.667 days?	11/12/24 8:00 AM	11/12/24 2:20 PM	Rand Mohamed;Lama Khalid;Ghada Abdullah
31	📅	Final Exam Presentation	21 days?	11/6/24 8:00 AM	12/4/24 5:00 PM	Rand Mohamed;Lama Khalid;Ghada Abdullah

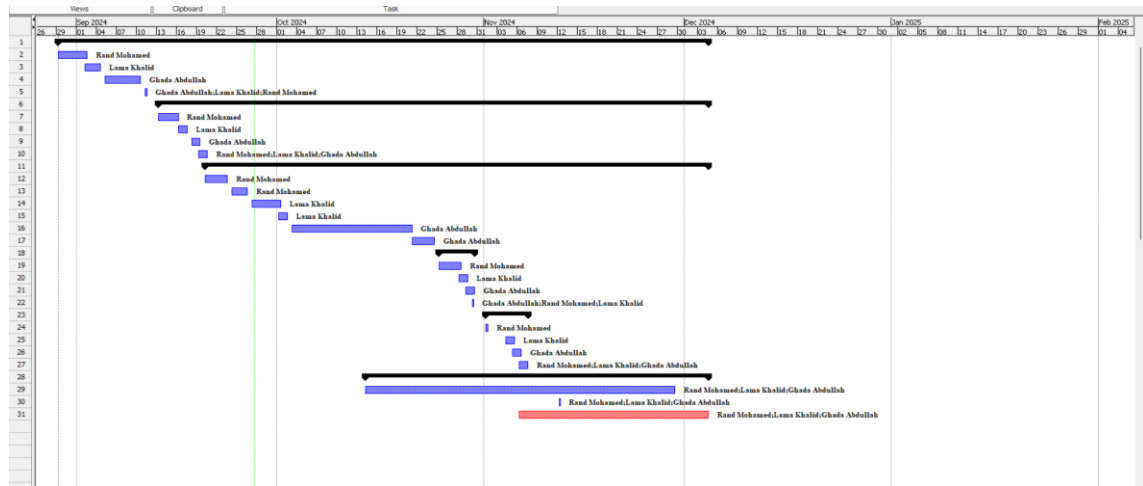


Figure 3.2: Gantt Chart

Chapter 4: Requirments and Analysis

Requirements and analysis involve determining the behaviors, features, contents, frameworks, and requirements of the system. This includes both functional and non-functional requirements, along with discussions of Unified Modeling Language (UML) models, scenarios, and diagrams. The analysis strategy focuses on exchanging abstract methods and ideas related to the system.

4.1 System Requirements

4.1.1 Functional Requirements

Functional requirements describe the primary goals of the system and its components, defining the expected behavior and outputs based on functional needs.

Processes of data collection and preparation:

1. Import the ICS-Flow dataset for anomaly detection analysis, which is specifically designed to test cybersecurity monitoring in ICS environments. It comprises raw network traffic captures of an industrial control system (ICS) subjected to a variety of simulated cyber-attacks, including but not limited to denial of service (DoS), man-in-the-middle (MITM), and malware infiltration. It contains all the phases of an attack to an ICS, from reconnaissance to exploitation of devices, and it is composed of traffic captures and logs generated by both control devices and security monitoring systems.
2. Preprocess raw data by cleaning, normalizing, and encoding it to ensure consistency in analysis.

Processes of anomaly detection:

1. The most suitable algorithm will be chosen for anomaly detection, either supervised or unsupervised, based on its performance in identifying patterns.

2. Use organized data to train models to recognize and improve detection accuracy for known threats.
3. In supervised mode, the system is trained on CAN message data where both normal and anomalous patterns are predefined. This allows the model to learn specific signatures of known attacks, enabling accurate detection of previously identified threats.
4. In unsupervised mode, the system detects deviations from expected CAN message patterns to identify unknown anomalies without prior labels.
5. Defining a Normal Behavior Profile: The system first establishes a profile of normal CAN message patterns by analyzing baseline data from the ICS. This normal behavior profile serves as a reference point for identifying deviations.
6. Threat Detection: The system detects threats by identifying unusual CAN messages that differ from the established normal behavior profile, alerting the professionals operating the ICS system to potential attacks.

Processes of evaluation and model performance:

1. The system calculates and displays metrics such as accuracy and detection rate to evaluate model effectiveness.

Processes of result visualization and reporting:

1. The system executes the code in Google Colab and displays model outputs, performance metrics, and visualizations for user interpretation.
2. The system generates visual reports with charts (e.g., Receiver Operating Characteristic (ROC) curves, confusion matrices) to display detection accuracy and performance metrics.

4.1.2 Non-Functional Requirements

The non-functional requirements outline how the system operates, detailing its qualities for performance assessment:

- **RELIABILITY:** The system provides a dependable environment that handles interruptions in data flow and

resumes processing without data loss.

- **USABILITY:** The system features a user-friendly interface that enables the ICS operators to interact with the system and interpret results without requiring deep technical knowledge.
- **SECURITY:** The system safeguards sensitive CAN network data, ensuring it remains secure and accessible only to authorized users.
- **PORTABILITY:** The system can be used in different industries.
- **SCALABILITY:** The system supports easy integration of models or algorithm modifications to adapt to evolving cybersecurity threats.

4.2 System Scenario

Scenarios help describe specific system functionalities during requirements analysis. Our system scenario focuses on developing a system to detect anomalous activities in ICS that utilizes CAN, given that ICS has become a target for cyberattacks.

The process begins by collecting CAN-based datasets, such as the ICS-Flow dataset, which includes genuine CAN messages and simulated attacks like ransomware, DoS, spoofing, and fuzzy attacks. This data, comprises training and testing sets. The training set is used to develop the model, while the testing set evaluates its effectiveness post-training. The algorithm selection step enables the choice between supervised learning, where the model learns from labelled attack data, and unsupervised learning, where it detects unusual patterns without prior labels. Then, the results are presented through scripts on Google Colab, allowing for interactive execution and analysis.

4.3 System Models

4.3.1 Use Case Diagram

The Use Case Diagram graphically represents the system's basic requirements. It specifies the behavior (what) and the method for achieving it (how). The primary purpose of use case modeling is to build a system from the end-user's perspective.

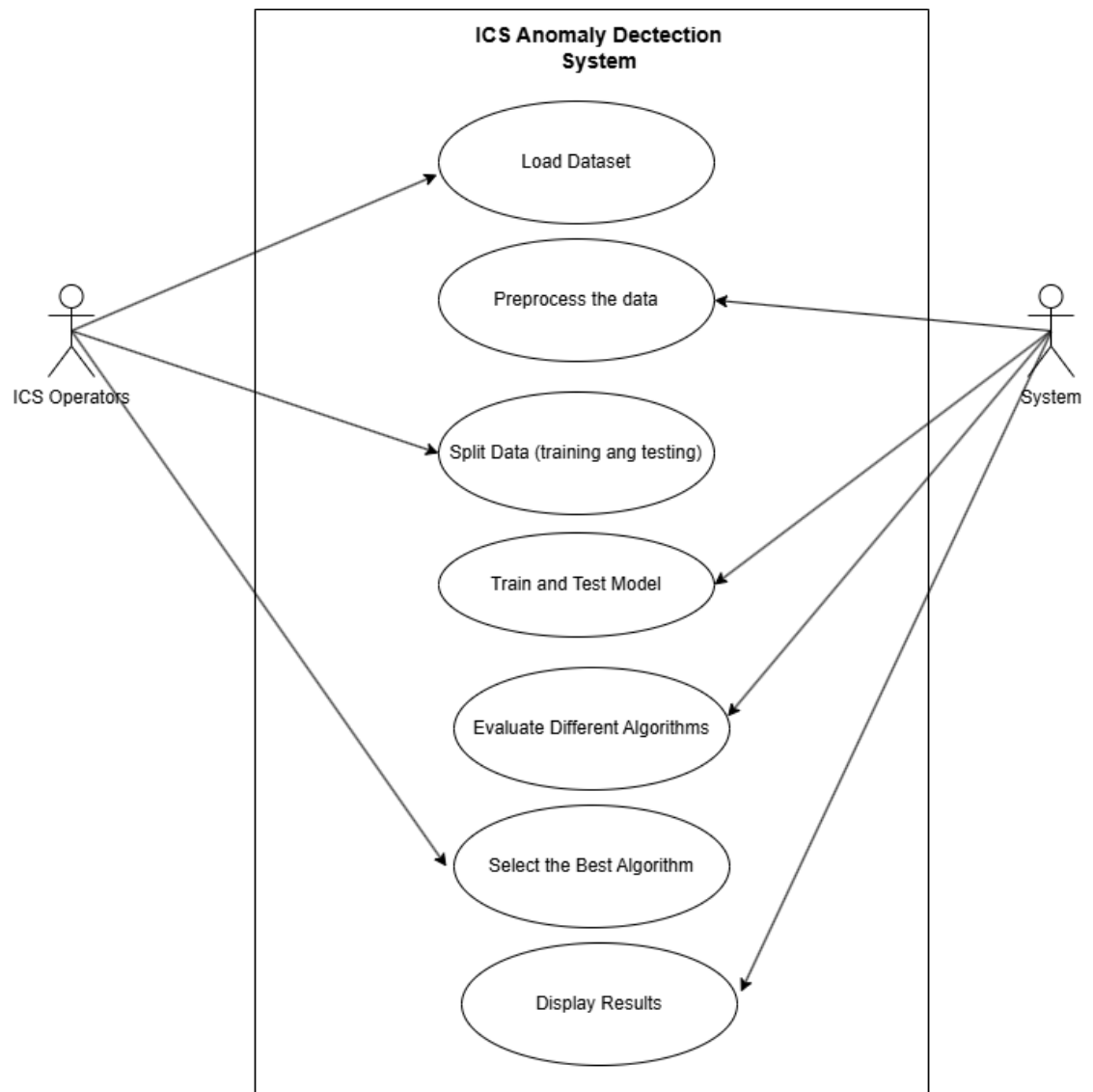


Figure 4.1: Use Case Diagram

4.3.2 Sequence Diagram

The Sequence Diagram is a dynamic graph that describes the collaboration and interactions between components within the system over time and the relationship between items.

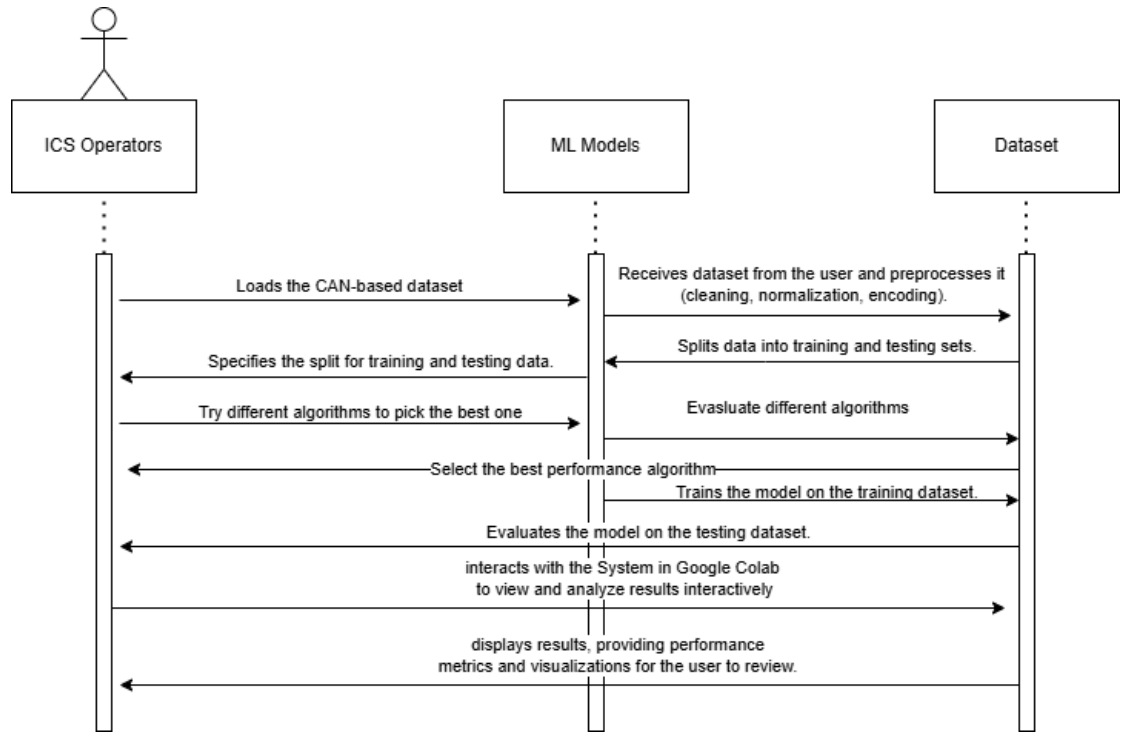


Figure 4.2: Sequence Diagram

4.3.3 Activity Diagram

The Activity Diagram is a flowchart used in software engineering to represent a system's dynamic aspects. It visualizes the workflow of a system or process, illustrating the sequence of actions and decisions that occur.

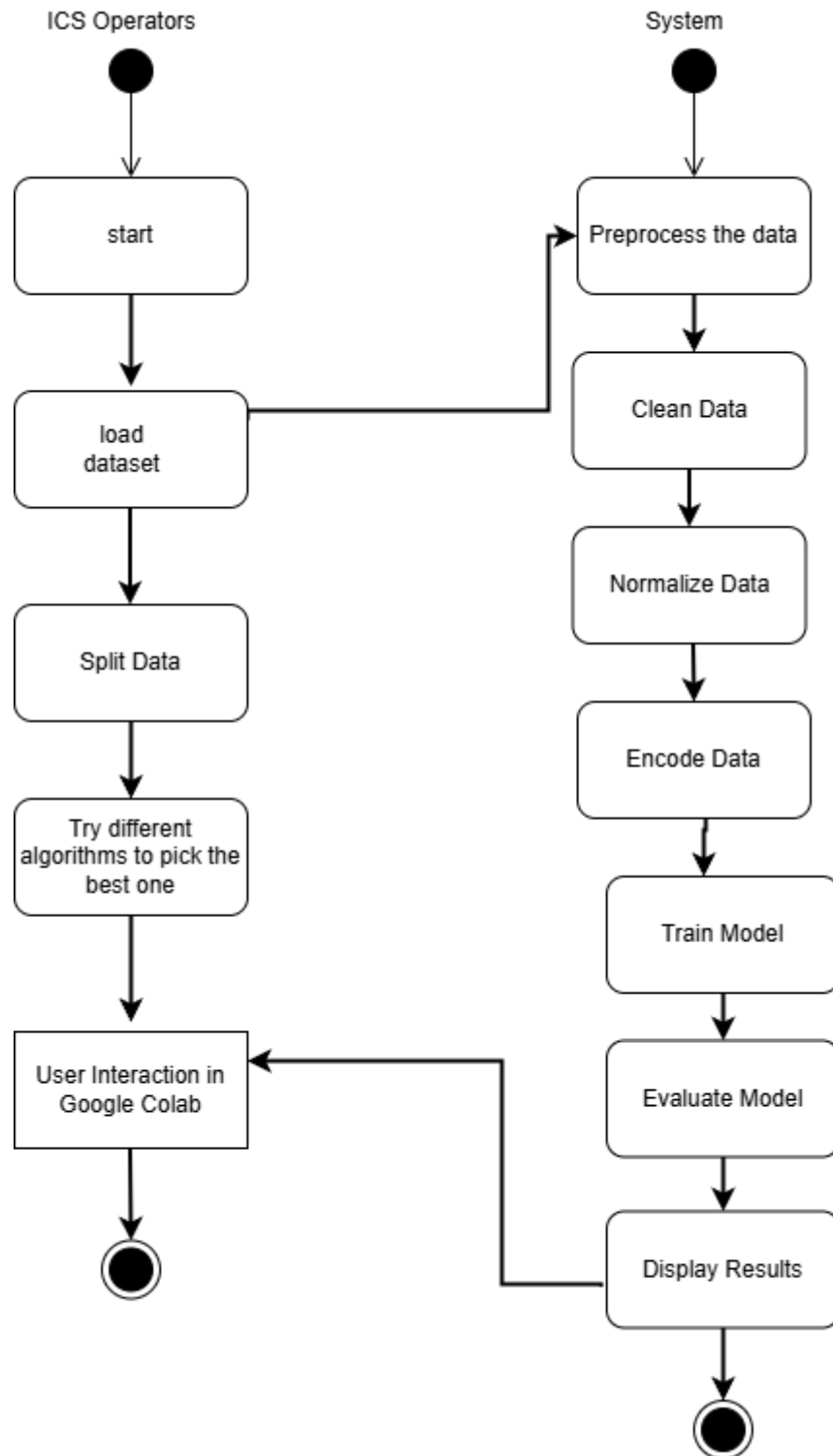


Figure 4.3: Activity Diagram

Chapter 5: System Design

System design involves creating a system's architecture, components, modules, and interfaces to ensure all parts function seamlessly. For our anomaly detection system in CAN-based Industrial Control Systems (ICS), the design focuses on structuring elements to facilitate seamless communication and efficient detection of cyber threats.

5.1 Product Features

For the CAN-based Industrial Control System (ICS) anomaly detection system, the following are the key features, main commands, and protocol messages. This setup is intended to implement the key features of a CAN-based ICS anomaly detection system in Google Colab, utilizing the ICS-Flow dataset and leveraging Python along with machine learning libraries such as Scikit-Learn and Keras.

1. Data Collection:

ICS-Flow Resource: The ICS-Flow dataset is a valuable resource for researchers and professionals focused on developing, testing, and benchmarking new cyber-physical security monitoring and detection solutions. It includes raw network traffic captures from an industrial control system (ICS) exposed to various simulated cyber-attacks, such as DoS, MITM, and malware infiltration. ICS-Flow features output from two widely used open-source security monitoring tools, OSSIM (Open-Source Security Information Management) and Suricata, providing insights into the detection and analysis capabilities of existing security frameworks against these threats [26].

- **Description:** The system collects and processes CAN-based data from the ICS-Flow dataset, including standard CAN messages and anomalous patterns from simulated attacks (e.g. DoS, spoofing, and fuzzy attacks).
- **Input:** Refer to Figure 5.1 CAN messages serve as the system's primary data input the dataset is uploaded and loaded into the Colab environment.


```

# Step 1: Upload the dataset
from google.colab import files

# Upload files (Assume the dataset is in CSV format)
uploaded = files.upload()

# Step 2: Load the dataset
import pandas as pd

# Load dataset (Replace 'dataset.csv' with your actual file name)
df = pd.read_csv('dataset.csv')
print("Data Loaded:")
print(df.head())

```

Figure 5.1: Code of Upload the dataset

2. Data Preprocessing

- Description: Refer to Figure 5.2, The system preprocesses incoming CAN messages by cleaning and formatting the data, converting it into a structure suitable for machine learning models. This may include data normalization, encoding CAN message IDs, and filtering out irrelevant data.

```

># Step 1: Encode categorical data if necessary
>df = pd.get_dummies(df, columns=['CAN_ID']) # Assuming CAN_ID is a categorical column
>print("Data Preprocessing Completed:")
>print(df.head())
>

```

Figure 5.2: Code of Data Processing

3. Anomaly Detection:

- Description: By using machine learning, the system models the normal behavior of the CAN network and then employs machine learning algorithms to analyze CAN messages and detect deviations from that normal behavior. Refer to figure 5.3, the system splits the dataset into training and testing sets. It trains models on extracted features using techniques such as Random Forest (RF), Isolation Forest to learn these normal behavior patterns and using training and testing sets. For example, if a specific CAN ID typically sends messages every 100 milliseconds, any deviations beyond this range could indicate anomalies.

- ML Models:
 - Supervised Detection: The system detects known attack patterns using a labeled training set of data.

Strengths:

- High Accuracy with Labeled Data: Supervised algorithms, such as Random Forests can achieve high accuracy when trained on labeled datasets that include both normal and attack patterns.
- Specific Attack Detection: When the types of potential attacks are known (e.g., DoS, spoofing), supervised models can be highly effective at identifying them, as they have been trained on labeled examples of these attacks.
- This is achieved through the use of training and testing sets, allowing for model evaluation and accuracy display, as shown in Figure 5.4.

```
)# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Figure 5.3: Code of training and testing sets

```
# Train the model
history = model.fit(X_train, y_train, epochs=10, batch_size=32, validation_data=(X_test, y_test))

# Evaluate the model
loss, accuracy = model.evaluate(X_test, y_test)
print(f"Supervised Model Accuracy: {accuracy}")
```

Figure 5.4: Code of Train and Evaluate the model

- Unsupervised Detection: This approach is used when data is unlabeled, aiming to detect anomalies based on deviations from normal behavior.

Strengths:

- No Need for Labeled Data: Unsupervised algorithms, such as Isolation Forest, One-Class SVM, and clustering methods (e.g., K-Means), do not require labeled data, making them suitable for ICS environments where labels may be scarce or unavailable.

- Flexibility and Adaptability: Since ICS environments can change over time, unsupervised models are often more flexible and adaptable to variations in normal behavior, allowing for effective anomaly detection even as conditions evolve.

5.2 Project Implementation

We will use Google Colab, shown in Figure 5.5, to provide an intuitive experience for various industries while showcasing the effectiveness of the model and its findings.

5. Setting Up Google Colab Environment: Import the required libraries for data processing, machine learning, and evaluation tools.
6. Upload the dataset: Upload the ICS-Flow dataset, which contains CAN messages and cyber-attack simulations.
7. Data Preparation: Prepare the data for model training by cleaning and transforming it into a suitable format.
8. Model Selection and Training: Select and train the model using a supervised algorithm, such as RandomForestClassifier, or an unsupervised algorithm, such as Isolation Forest clustering.
9. Model Evaluation: Evaluate the performance of each algorithm using metrics such as accuracy, confusion matrix, and classification report, allowing for the selection of the best algorithm.
10. Interactive Analysis on Google Colab: Visualize the patterns and anomalies in CAN messages by plotting data.

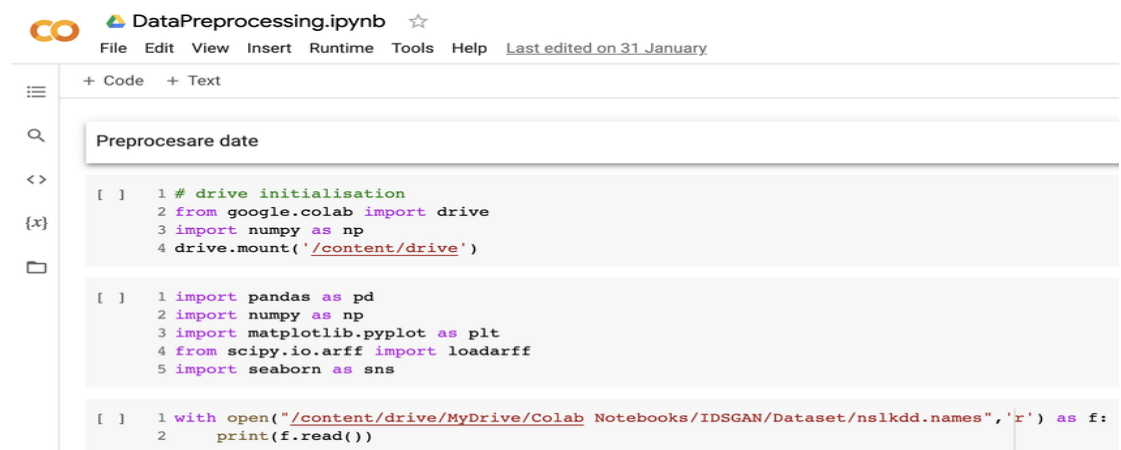


Figure 5.5: Screen capture of the Google Colab interface.[25]

5.3 High Level Design

5.3.1 Dataflow Diagram

Dataflow diagram (DFD) visually represents how information flows through a process or system. It shows data inputs, outputs, and the paths between each destination using standard symbols such as rectangles, circles, and arrows. Refer to Figure 5.6, the steps of our Dataflow Diagram are as follows:

1. ICS Operator:

The process starts with the ICS operator which initiates the workflow.

2. Dataset File (D1):

The dataset file serves as the starting input for the process.

3. Steps in Data Processing and Model Development:

- Step 1: Load Dataset

The dataset file is loaded into the system.

- Step 2: Preprocess the Data

The data undergoes preprocessing to clean and prepare it for analysis.

- Step 3: Split Data

The dataset is divided into training and testing sets.

- Step 4: Train and Test Model

Machine learning models are trained and tested using the prepared dataset.

- Step 5: Try Different Algorithms

Multiple algorithms are tested to determine which performs best.

- Step 6: Evaluate the Algorithms

The performance of the algorithms is evaluated based on selected metrics.

- Step 7: Select the Best Algorithm
The best-performing algorithm is chosen for deployment.
- Step 8: Display Results
The outcomes of the evaluation are visualized and displayed.

4. System:

The results or outputs are integrated into the ICS system for practical application.

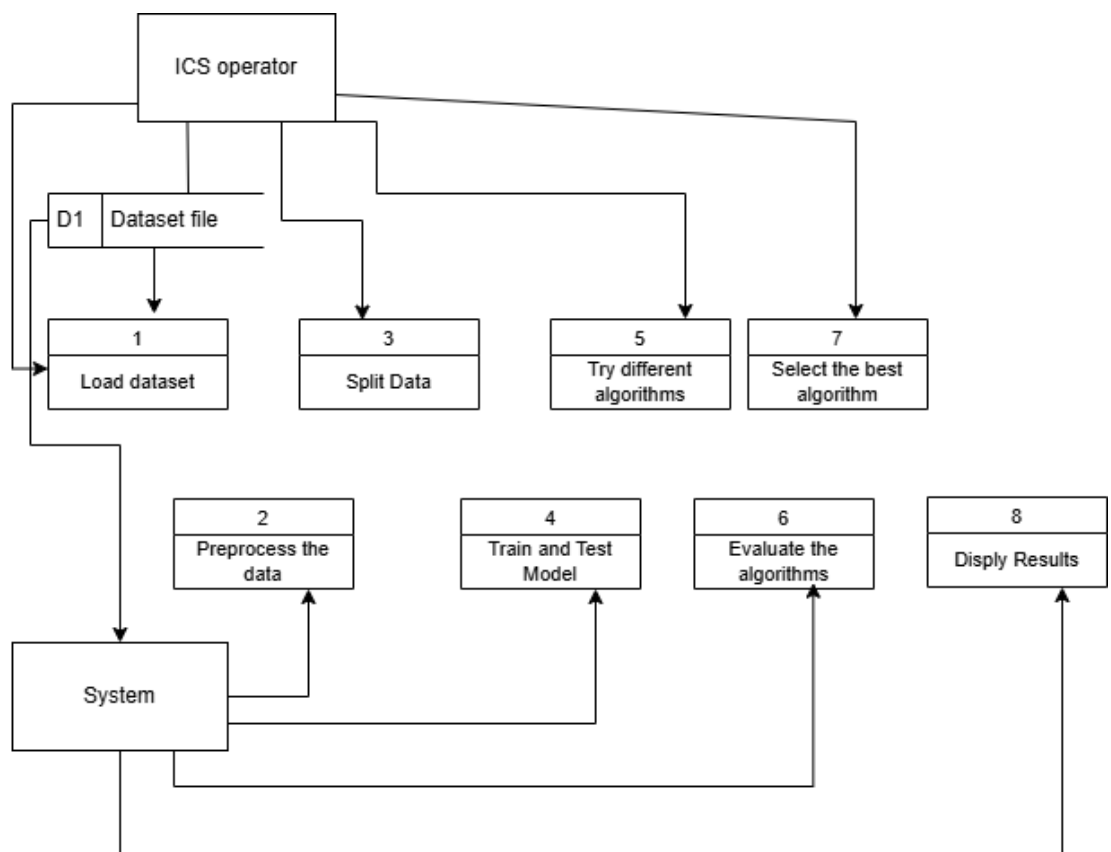


Figure 5.6: Dataflow Diagram

Chapter 6: PROJECT IMPLEMENTATION

This chapter outlines the implementation of the ICS anomaly detection project, including the choice of programming language, tools, and platform. It also discusses the coding approach, verification through testing, and validation against user requirements.

For this project, we selected three machine learning algorithms for anomaly detection: Random Forest, Isolation Forest, and Support Vector Machine (SVM):

- Random Forest (RF): A robust ensemble learning method that builds multiple decision trees and aggregates their results, making it effective for distinguishing between normal and anomalous patterns [27].
- Isolation Forest (IF): Specifically designed for anomaly detection, IF isolates anomalies by randomly selecting features and partitioning the data, making it efficient in detecting outliers in ICS traffic [28].
- Support Vector Machine (SVM): A powerful supervised learning algorithm that excels in classification tasks, particularly in high-dimensional spaces. It operates by finding the optimal hyperplane that separates different classes in the feature space, maximizing the margin between the nearest data points (support vectors) of each class [30].

These algorithms complement each other, allowing us to compare supervised and unsupervised learning approaches for anomaly detection in industrial control systems.

6.1 Coding

The project is implemented using Python due to its extensive machine learning and data analysis libraries. The coding is done in Google Colab, which provides a cloud-based environment with built-in machine learning and GPU acceleration support.

The code snippet shown in Figure 6.1 is for setting up a machine learning project, particularly for data preprocessing, resampling, and visualization. Here's a breakdown of each part:

- Install essential Python libraries required for data handling, machine learning, and visualization.
- pandas and numpy: Used for data manipulation.
- scikit-learn: Provides tools for machine learning, including preprocessing, model training, and evaluation.
- imbalanced-learn: A library for handling imbalanced datasets, including SMOTE.
- matplotlib and seaborn: Used for data visualization.
- StandardScaler: Standardizes features by removing the mean and scaling to unit variance.
- MinMaxScaler: Scales data to a specific range, usually between 0 and 1.
- LabelEncoder: Converts categorical labels into numerical values.
- Resample: Used for data resampling, typically for handling imbalanced datasets.

```
 # Install required libraries
!pip install pandas numpy scikit-learn imbalanced-learn matplotlib seaborn
# Import libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder
from sklearn.utils import resample
from imblearn.over_sampling import SMOTE
import matplotlib.pyplot as plt
import seaborn as sns
```

Figure 6.1: Google Colab

ICS-Flow Dataset Overview

The ICS-Flow dataset is a labeled dataset specifically designed for anomaly detection in Industrial Control Systems (ICS). It consists of various network traffic features extracted from ICS environments, making it suitable for machine learning-based anomaly detection. The dataset contains multiple features representing different network and communication attributes in ICS environments [26]. These include:

- Source & Destination Addresses (sAddress, rAddress, sMACs, rMACs, sIPs, rIPs) – Identifiers for network communication.
- Traffic Timing & Durations (startDate, endDate, start, end, duration, startOffset, endOffset) – Captures when and how long the traffic occurred.
- Packet-Level Features (sPackets, rPackets, sBytesSum, rBytesSum, sBytesMax, rBytesMax, etc.) – Represents data transmission details.
- TCP/IP Flags & Rates (sAckRate, rAckRate, sFinRate, rFinRate, sSynRate, rSynRate, etc.) – Key indicators of network behaviors.

The dataset includes multiple label columns such as:

- ❖ IT_B_Label: Provides a binary classification of network traffic as either normal (0) or anomalous (1) based on Injection Timing methodology.
- ❖ IT_M_Label: Extends IT_B_Label to classify network traffic into multiple attack categories instead of just normal/anomalous. It provides finer granularity to distinguish different types of attacks.
- ❖ NST_B_Label: Another binary classification label based on Network Security Tools methodology.
- ❖ NST_M_Label: Similar to NST_B_Label, but instead of binary classification, it provides a multi-class categorization of attacks. It allows for a detailed classification of different cyber threats.

These labels indicate whether the network flow is benign (normal traffic) or affected (anomalous behavior) based on ICS security analysis.

Classification Type

Since the dataset is labeled, it is used for supervised learning approaches. Based on the label categories as shown in Figure 6.2, it can be used for:

- Binary Classification: If the labels are grouped into "Benign" vs. "Anomalous," the dataset is used for binary classification tasks.
- Multi-Class Classification: If different types of anomalies are considered separately (e.g., different attack types), then it is used for multi-class classification.

```
label_columns = ['IT_B_Label', 'IT_M_Label', 'NST_B_Label', 'NST_M_Label']

for col in label_columns:
    print(f"Unique values in {col}:")
    print(df[col].unique())
    print("-" * 40)
```

Unique values in IT_B_Label:
[0 1]

Unique values in IT_M_Label:
['Normal' 'port-scan' 'ip-scan' 'mitm' 'replay' 'ddos']

Unique values in NST_B_Label:
[0 1]

Unique values in NST_M_Label:
['Normal' 'port-scan' 'ip-scan' 'mitm' 'replay' 'ddos']

Figure 6.2: Label Categories

1. Data Preprocessing

Data Preprocessing involves several key steps to prepare the ICS-Flow dataset for analysis. First, load the dataset into a Pandas DataFrame as shown in Figure 6.3. Next, check for missing values and handle them through either imputation or removal. After that, convert categorical features into a numerical format where applicable. Finally, standardize the numerical features with StandardScaler to ensure uniformity across the dataset. This systematic approach enhances the dataset's quality and suitability for analysis.

```
import pandas as pd

# Load the dataset (adjust path if needed)
df = pd.read_csv('/content/sample_data/Dataset.csv')

# Display basic info
print(df.head()) # Show first few rows
print(df.info()) # Check data types
print(df.describe()) # Summary statistics
```

Figure 6.3: Load Dataset

2. Preprocessing data and Feature Selection

Based on the code in Figure 6.4, The selected categorical features for processing include:

- "IT_M_Label"
- "NST_M_Label"

These categorical labels are converted into numerical values using LabelEncoder, making them suitable for machine learning models.

```

→ Value counts for IT_B_Label:
IT_B_Label
0    30236
1    15482
Name: count, dtype: int64
-----

Value counts for IT_M_Label:
IT_M_Label
Normal      30236
replay      4300
ddos         4221
port-scan    3235
mitm         3014
ip-scan       712
Name: count, dtype: int64
-----

Value counts for NST_B_Label:
NST_B_Label
0    36706
1     9012
Name: count, dtype: int64
-----

Value counts for NST_M_Label:
NST_M_Label
Normal      36706
mitm        2584
replay      2358
port-scan    1944
ddos         1934
ip-scan       192
Name: count, dtype: int64
-----

```

Figure 6.4: All Numerical Columns

For numerical features we perform feature selection and preprocessing using StandardScaler, as shown in the code in Figure 6.5:

- Using `df.select_dtypes(include=['float64', 'int64']).columns`, we automatically select all numerical columns (i.e., features with float64 or int64 data types).
- `StandardScaler()` is used to normalize the data ensuring that all features have a zero mean and unit variance, which helps improve model stability and performance.

- By using `df.drop_duplicates()`, we ensure that duplicate rows do not negatively impact training.

```
from sklearn.preprocessing import StandardScaler

# Select numerical columns
num_cols = df.select_dtypes(include=['float64', 'int64']).columns

# Normalize numerical features
scaler = StandardScaler()
df[num_cols] = scaler.fit_transform(df[num_cols])

df = df.drop_duplicates()
```

Figure 6.5: Feature Selection

3. Dataset splitting

As illustrated in Figure 6.6, The dataset is first separated into independent variables (X) and the dependent variable (y). The independent variables (X) include all features except IT_M_Label, which serves as the target variable (y) for classification. Next, the data is split into training and testing sets using the `train_test_split` function:

- 80% of the data is allocated to the training set (X_train, y_train) to train the model.
- 20% of the data is reserved for the testing set (X_test, y_test) to evaluate the model's performance.
- Setting `random_state=42` ensures reproducibility, meaning the split remains consistent across different runs.

To improve model performance, feature scaling is applied using `StandardScaler()` to prevent features with larger numerical ranges from dominating the learning process. The `fit_transform(X_train)` method learns scaling parameters from the training data and applies the transformation, while `transform(X_test)` ensures the same scaling is applied to the test data without re-fitting, thereby preventing data leakage.

```

# Feature selection (assuming 'IT_M_Label' is the target column for multi-class classification)
X = ics_data.drop(columns=['IT_M_Label']) # Features
y_multi = ics_data['IT_M_Label'] # Multi-class target

# Convert multi-class labels to binary labels for Isolation Forest (0 = Normal, 1 = Malicious)
y_binary = np.where(y_multi == 0, 0, 1) # Treat all non-zero labels as malicious

# Split dataset
X_train, X_test, y_train_multi, y_test_multi = train_test_split(X, y_multi, test_size=0.2, random_state=42)
X_train_bin, X_test_bin, y_train_bin, y_test_bin = train_test_split(X, y_binary, test_size=0.2, random_state=42)

# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
X_train_bin = scaler.fit_transform(X_train_bin)
X_test_bin = scaler.transform(X_test_bin)

```

Figure 6.6: Split Dataset

4. Anomaly Detection Implementation

In this section, we explore three approaches to anomaly detection: Isolation Forest, Random Forest and SVM. The following subsections detail the implementation of each algorithm, including their performance metrics and visualization techniques for better understanding the results.

In this project, multiple machine learning algorithms were implemented for anomaly detection in Industrial Control Systems (ICS). Each algorithm was applied in two different modes:

- The first mode uses binary classification to determine whether the observed network traffic is normal or anomalous.
- The second mode uses multi-class classification to identify the specific type of attack if an anomaly is detected.

The following algorithms were evaluated in both binary and multi-class settings:

- Random Forest
- Support Vector Machine (SVM)
- Isolation Forest

This dual approach enables a thorough performance evaluation of each model in both general anomaly detection and detailed attack identification.

1. Isolation Forest

Isolation Forest is an unsupervised anomaly detection algorithm. We implement Isolation Forest with a specified contamination rate to detect anomalies, as illustrated in Figures 6.8 and 6.9, which demonstrate the implementation and evaluate its performance. Initially, we applied the Isolation Forest algorithm using a multi-class classification approach based on the IT_M_Label. However, the results were unsatisfactory in terms of accuracy and class-wise performance. As shown in the classification report and confusion matrix, the model struggled to correctly distinguish between multiple classes correctly, achieving an overall accuracy of only 5.51% as shown in Figure 6.7. To address this limitation, we redefined the problem as a binary classification task. Specifically, we converted the labels into two classes:

- 0 for normal instances
- 1 for all types of anomalies (i.e., any label not equal to 0)

```
=== Isolation Forest (Multi-class IT_M_Label) ===
Accuracy: 0.0551
```

	precision	recall	f1-score	support
0	0.01	0.00	0.00	5879
1	0.06	0.51	0.11	872
2	0.00	0.00	0.00	91
3	0.00	0.00	0.00	444
4	0.00	0.00	0.00	251
5	0.00	0.00	0.00	735
accuracy			0.06	8272
macro avg	0.01	0.09	0.02	8272
weighted avg	0.01	0.06	0.01	8272

Figure 6.7: Isolation Forest First Multi-class mode

Below is a breakdown of each step:

Key Parameters:

- `n_estimators=100`: Uses 100 base estimators (trees).
- `contamination=0.1`: Assumes 10% of the data contains anomalies.
- `random_state=42`: Ensures reproducibility.

Functions used in the code:

- `fit(X_train)`: The model learns the patterns from the training data.
- `iso_forest.predict()`: Converts predictions to match our labels (0 = Normal, 1 = Malicious).
- `predict(X_test)`: Classifies test samples as:
 - ❖ 1 → Normal
 - ❖ -1 → Anomaly

To convert labels to binary format:

- ❖ 1 → 0 (Normal)
- ❖ -1 → 1 (Anomaly)

Performance Metrics:

- `accuracy_score(y_test, iso_preds)`: Calculates the accuracy of predictions.
- `classification_report(y_test, iso_preds)`: Prints precision, recall, and F1-score such that:
 - ❖ $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$: Indicates how many predicted anomalies were correct.
 - True Positives (TP): The number of actual anomalies that were correctly predicted as anomalies.
 - False Positives (FP): The number of normal instances incorrectly predicted as anomalies (false alarms).
 - ❖ $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$: Measures how many actual anomalies were detected.

- ❖ $F1\text{-score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$: Balances precision and recall into a single score.

Confusion Matrix:

The confusion matrix shows the number of correct and incorrect predictions, converts predictions to match our labels (0 = Normal, 1 = Malicious). The following steps compute and visualize the confusion matrix:

- `confusion_matrix(y_test, iso_preds)`: Computes the confusion matrix between the actual label (`y_test`) and the predicted label (`iso_preds`) from the model.
- `plt.figure(figsize=(6, 4))`: Sets the figure size for the plot.
- `sns.heatmap(..., annot=True, fmt='d', cmap='Oranges')`: Displays the confusion matrix as a heatmap using Seaborn, with annotations for the values inside the matrix.
- `plt.xlabel('Predicted')`: Labels the x-axis as Predicted Values.
- `plt.ylabel('Actual')`: Labels the y-axis as Actual Values.
- `plt.title('Isolation Forest Confusion Matrix')`: Assigns a title to the plot.
- `plt.show()`: Displays the final confusion matrix plot.

```
# ===== ISOLATION FOREST 1: Multi-class from IT_M_Label =====
X_iso1 = ics_data.drop(columns=['IT_M_Label', 'IT_B_Label'])
y_iso1 = np.where(ics_data['IT_M_Label'] == 0, 0, 1)

X_train_iso1, X_test_iso1, y_train_iso1, y_test_iso1 = train_test_split(X_iso1, y_iso1, test_size=0.2, random_state=42)
scaler_iso1 = StandardScaler()
X_train_iso1 = scaler_iso1.fit_transform(X_train_iso1)
X_test_iso1 = scaler_iso1.transform(X_test_iso1)

iso_model1 = IsolationForest(n_estimators=100, contamination=0.1, random_state=42)
iso_model1.fit(X_train_iso1)
y_pred_iso1 = np.where(iso_model1.predict(X_test_iso1) == 1, 0, 1)

accuracy_iso1 = accuracy_score(y_test_iso1, y_pred_iso1)
comparison_df.append({'Model': 'Isolation Forest (Binary IT_M_Label)', 'Accuracy': accuracy_iso1})

print("\n==== Isolation Forest (Binary IT_M_Label) ====")
print(f"Accuracy: {accuracy_iso1:.4f}")
print(classification_report(y_test_iso1, y_pred_iso1))
sns.heatmap(confusion_matrix(y_test_iso1, y_pred_iso1), annot=True, fmt='d', cmap='Oranges')
plt.title("Confusion Matrix - Isolation Forest (IT_M_Label)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```


Figure 6.8: Isolation Forest Multi-class mode

```
# ===== ISOLATION FOREST 2: IT_B_Label (binary) =====
X_iso2 = ics_data.drop(columns=['IT_M_Label', 'IT_B_Label'])
y_iso2 = ics_data['IT_B_Label']

X_train_iso2, X_test_iso2, y_train_iso2, y_test_iso2 = train_test_split(X_iso2, y_iso2, test_size=0.2, random_state=42)
scaler_iso2 = StandardScaler()
X_train_iso2 = scaler_iso2.fit_transform(X_train_iso2)
X_test_iso2 = scaler_iso2.transform(X_test_iso2)

iso_model2 = IsolationForest(n_estimators=100, contamination=0.1, random_state=42)
iso_model2.fit(X_train_iso2)
y_pred_iso2 = np.where(iso_model2.predict(X_test_iso2) == 1, 0, 1)

accuracy_iso2 = accuracy_score(y_test_iso2, y_pred_iso2)
comparison_df.append({'Model': 'Isolation Forest (IT_B_Label)', 'Accuracy': accuracy_iso2})

print("\n==== Isolation Forest (Binary IT_B_Label) =====")
print(f"Accuracy: {accuracy_iso2:.4f}")
print(classification_report(y_test_iso2, y_pred_iso2))
sns.heatmap(confusion_matrix(y_test_iso2, y_pred_iso2), annot=True, fmt='d', cmap='Greens')
plt.title("Confusion Matrix - Isolation Forest (IT_B_Label)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Figure 6.9: Isolation Forest Binary mode

2. Random Forest

We implement Random Forest as a supervised classification approach, as illustrated in Figures 6.10 and 6.11. The provided code trains a Random Forest Classifier on the dataset, evaluates its performance, and visualizes the results using a confusion matrix. Key aspects include:

- RandomForestClassifier is initialized with 100 decision trees (`n_estimators=100`) to form an ensemble model.
- `random_state=42` ensures reproducibility.
- The model is trained using `X_train` (features) and `y_train` (labels).
- The trained model is used to predict the class labels on the test dataset (`X_test`).
- The output `y_pred` contains the predicted labels, and the following metrics are computed:
 - `accuracy_score(y_test, y_pred)`: computes the overall accuracy of the model.
 - `classification_report(y_test, y_pred)`: generates a performance report includes:
 - Precision (How many predicted positives were actually positive)

- Recall (How many actual positives were correctly predicted)
 - F1-score (Balance between precision and recall)
 - Support (Number of actual occurrences for each class)
- The `confusion_matrix(y_test, y_pred)` function generates a matrix comparing actual vs. predicted labels. This matrix helps identify misclassifications, such as false positives (FP) and false negatives (FN).
 - `sns.heatmap()` is used to display the confusion matrix as a heatmap, with the x-axis (Predicted) and y-axis (Actual) representing the classification results.

```
# ===== RANDOM FOREST (MULTI-CLASS with IT_M_Label) =====
X_rf = ics_data.drop(columns=['IT_M_Label', 'IT_B_Label'])
y_rf = ics_data['IT_M_Label']

X_train_rf, X_test_rf, y_train_rf, y_test_rf = train_test_split(X_rf, y_rf, test_size=0.2, random_state=42)
scaler_rf = StandardScaler()
X_train_rf = scaler_rf.fit_transform(X_train_rf)
X_test_rf = scaler_rf.transform(X_test_rf)

rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train_rf, y_train_rf)
y_pred_rf = rf_model.predict(X_test_rf)

accuracy_rf = accuracy_score(y_test_rf, y_pred_rf)
comparison_df.append({'Model': 'Random Forest (IT_M_Label)', 'Accuracy': accuracy_rf})

print("\n=== Random Forest (Multi-class IT_M_Label) ===")
print(f"Accuracy: {accuracy_rf:.4f}")
print(classification_report(y_test_rf, y_pred_rf))
sns.heatmap(confusion_matrix(y_test_rf, y_pred_rf), annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix - Random Forest")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Figure 6.10: Random Forest (RF) for Multi-class Mode

```
# ===== RANDOM FOREST (BINARY with IT_B_Label) =====
X_rf_bin = ics_data.drop(columns=['IT_M_Label', 'IT_B_Label'])
y_rf_bin = ics_data['IT_B_Label']

X_train_rf_bin, X_test_rf_bin, y_train_rf_bin, y_test_rf_bin = train_test_split(X_rf_bin, y_rf_bin, test_size=0.2, random_state=42)
scaler_rf_bin = StandardScaler()
X_train_rf_bin = scaler_rf_bin.fit_transform(X_train_rf_bin)
X_test_rf_bin = scaler_rf_bin.transform(X_test_rf_bin)

rf_bin_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_bin_model.fit(X_train_rf_bin, y_train_rf_bin)
y_pred_rf_bin = rf_bin_model.predict(X_test_rf_bin)

accuracy_rf_bin = accuracy_score(y_test_rf_bin, y_pred_rf_bin)
comparison_df.append({'Model': 'Random Forest (Binary IT_B_Label)', 'Accuracy': accuracy_rf_bin})

print("\n=== Random Forest (Binary IT_B_Label) ===")
print(f"Accuracy: {accuracy_rf_bin:.4f}")
print(classification_report(y_test_rf_bin, y_pred_rf_bin))
sns.heatmap(confusion_matrix(y_test_rf_bin, y_pred_rf_bin), annot=True, fmt='d', cmap='BuGn')
plt.title("Confusion Matrix - Random Forest (Binary IT_B_Label)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Figure 6.11: Random Forest (RF) for Binary Mode

3. SVM

Support Vector Machine (SVM) is a powerful supervised learning algorithm used for classification tasks. The dataset `ics_data`, as shown in Figure 6.11, includes multiple features and two labels: `IT_M_Label` for multi-class classification and `IT_B_Label` for binary classification.

The first part of the code applies SVM using the multi-class classification approach to identify the type of attack in the ICS dataset. Here, all columns except `IT_M_Label` and `IT_B_Label` are used as features (`X_svm_mc`), with `IT_M_Label` serving as the target label (`y_svm_mc`). The data is then split into training and testing sets using `train_test_split`. To improve model performance, the features are standardized using `StandardScaler`. An SVM model with a radial basis function (RBF) kernel is trained on the scaled training data. After making predictions on the test set, the model's accuracy is calculated, and a detailed classification report is generated. A heatmap of the confusion matrix is plotted to visualize the performance in identifying different types of attacks.

In the second part, the same process is repeated using binary classification with `IT_B_Label` as the target, categorizing the data as either

normal or anomalous, as shown in Figure 6.12. The features (X_svm_bin) are prepared by removing both labels from the dataset, with y_svm_bin set as the binary label. The data undergoes the same steps: splitting, scaling, training the SVM model, and evaluating the predictions. The model's performance is again measured using accuracy, with results visualized through a classification report and a confusion matrix heatmap. Both multi-class and binary results are stored in comparison_df for comparative analysis across models.

```
# ===== SVM - MULTI-CLASS (IT_M_Label) =====
X_svm_mc = ics_data.drop(columns=['IT_M_Label', 'IT_B_Label'])
y_svm_mc = ics_data['IT_M_Label']

X_train_svm_mc, X_test_svm_mc, y_train_svm_mc, y_test_svm_mc = train_test_split(X_svm_mc, y_svm_mc, test_size=0.2, random_state=42)
scaler_svm_mc = StandardScaler()
X_train_svm_mc = scaler_svm_mc.fit_transform(X_train_svm_mc)
X_test_svm_mc = scaler_svm_mc.transform(X_test_svm_mc)

svm_mc_model = SVC(kernel='rbf')
svm_mc_model.fit(X_train_svm_mc, y_train_svm_mc)
y_pred_svm_mc = svm_mc_model.predict(X_test_svm_mc)

accuracy_svm_mc = accuracy_score(y_test_svm_mc, y_pred_svm_mc)
comparison_df.append({'Model': 'SVM (Multi-class IT_M_Label)', 'Accuracy': accuracy_svm_mc})

print("\n=== SVM (Multi-class IT_M_Label) ===")
print(f"Accuracy: {accuracy_svm_mc:.4f}")
print(classification_report(y_test_svm_mc, y_pred_svm_mc))
sns.heatmap(confusion_matrix(y_test_svm_mc, y_pred_svm_mc), annot=True, fmt='d', cmap='Purples')
plt.title("Confusion Matrix - SVM Multi-class")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Figure 6.12: SVM for Multi-class mode

```
# ===== SVM - BINARY (IT_B_Label) =====
X_svm_bin = ics_data.drop(columns=['IT_M_Label', 'IT_B_Label'])
y_svm_bin = ics_data['IT_B_Label']

X_train_svm_bin, X_test_svm_bin, y_train_svm_bin, y_test_svm_bin = train_test_split(X_svm_bin, y_svm_bin, test_size=0.2, random_state=42)
scaler_svm_bin = StandardScaler()
X_train_svm_bin = scaler_svm_bin.fit_transform(X_train_svm_bin)
X_test_svm_bin = scaler_svm_bin.transform(X_test_svm_bin)

svm_bin_model = SVC(kernel='rbf')
svm_bin_model.fit(X_train_svm_bin, y_train_svm_bin)
y_pred_svm_bin = svm_bin_model.predict(X_test_svm_bin)

accuracy_svm_bin = accuracy_score(y_test_svm_bin, y_pred_svm_bin)
comparison_df.append({'Model': 'SVM (Binary IT_B_Label)', 'Accuracy': accuracy_svm_bin})

print("\n=== SVM (Binary IT_B_Label) ===")
print(f"Accuracy: {accuracy_svm_bin:.4f}")
print(classification_report(y_test_svm_bin, y_pred_svm_bin))
sns.heatmap(confusion_matrix(y_test_svm_bin, y_pred_svm_bin), annot=True, fmt='d', cmap='Reds')
plt.title("Confusion Matrix - SVM Binary")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Figure 6.13: SVM for Binary mode

6.2 Verification

The verification phase involves rigorous testing of the implemented anomaly detection models to ensure their accuracy and reliability.

Unit Testing

Unit testing is a critical step in ensuring the correctness of data preprocessing and model predictions. It helps validate that each component functions as expected before integration into a complete pipeline. In our project, we focus on the following:

- Verifying that data preprocessing steps were correctly applied.
- Ensuring that feature scaling and categorical encoding work as expected.
- Checking the correctness of model predictions by running sample test cases.

System Testing

System testing evaluates model accuracy using several metrics, including Precision, Recall, F1-score. The dataset contains 64 features and multiple label columns, including IT_M_Label, NST_B_Label, and NST_M_Label, which categorize normal and anomalous instances. While high accuracy is desirable, it is essential to examine other metrics (precision, recall, and F1-score) to ensure the model performs well across all classes.

6.3 Validation

Validation involves comparing the results with known attack labels in the dataset to evaluate whether the system meets the expected requirements.

Validation Approach:

1. Manual Review:
 - Check whether detected anomalies align with documented attacks.
 - Verify that normal traffic is classified correctly.
2. User Requirements Validation:

- Ensure that the model can effectively differentiate between normal and anomalous ICS traffic.

Chapter 7: PROJECT TESTING (EVALUATION)

This chapter outlines the evaluation and testing process of the anomaly detection system developed for CAN-based Industrial Control Systems (ICS), using Isolation Forest, Random Forest, and SVM algorithms. The primary objective of the testing phase is to assess the performance, reliability, and effectiveness of the proposed solution in identifying and classifying anomalies within the ICS dataset. The evaluation aims to validate whether the system meets the predefined objectives and expectations, focusing on binary classification of anomalies (malicious vs. normal) and multi-class classification of attack types. The findings are critically analyzed to determine the extent to which the project objectives were achieved.

7.1 Metrics

The evaluation metrics used to measure the performance of the anomaly detection system include:

1. Accuracy: Measures the proportion of correctly classified instances out of the total instances.

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Instances}}$$

2. F1-score: The harmonic mean of precision and recall.

$$F1 = 2 * \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

3. Macro F1-score: Takes the F1-score of each class separately and averages them. It gives equal importance to all classes (even if some classes have fewer instances).

$$\text{Macro F1 score} = \frac{1}{N} \sum_{i=1}^N F1_i$$

Where N is the number of classes, and $F1_i$ is the F1 score of class i.

4. Weighted Avg F1-score: Similar to Macro F1-score but is weighted by the number of instances in each class.

$$\text{Weighted Avg F1} = \sum \left(\frac{\text{F1 - score of class} \times \text{Number of instances in class}}{\text{Total Instances}} \right)$$

5. Misclassification of Minority Classes: Analyzes the model's ability to correctly identify less represented classes, specifically focusing on false negatives in the malicious category

7.2 Experimental Setup

The experimental setup utilized the ICS-Flow dataset, which was preprocessed using standard techniques, including label encoding and feature scaling. The dataset was split into training and testing sets in an 80/20 ratio. The experiments were conducted in a controlled environment using Google Colab, leveraging Python's Scikit-learn library for implementation.

7.3 The Experiments

7.3.1 Participants

The testing was not participant-based as it focused on system performance using historical data. However, the sample size included all valid data points within the ICS-Flow dataset after preprocessing.

7.3.2 Treatments

The experiment included running all models on the test data, capturing predictions, and comparing them against actual labels using confusion matrices and classification reports.

7.3.3 Procedures

The evaluation process included the following steps:

1. Preprocessing the dataset by encoding categorical features and scaling numerical features.
2. Splitting the data into training and testing subsets.
3. Training the Isolation Forest, Random Forest and SVM models for both anomaly detection (binary classification) and attack type classification (multi-class classification).
4. Evaluating all models using accuracy, F1-scores, and confusion matrices.
5. Visualizing performance metrics through heatmaps and performance comparison tables.

7.3.4 Data Analysis

Data analysis involved:

- Confusion Matrix: Visualized true positives, false positives, true negatives, and false negatives.
- Classification Report: Provided detailed performance metrics, including precision, recall, and F1-scores.
- Comparison Analysis: Evaluated the model's performance against benchmarks and similar anomaly detection systems.

7.4 Results

7.4.1 Isolation Forest

The Isolation Forest model achieves an accuracy of 0.81, This means the model correctly classified 81% of the data, as illustrated in Figures 7.1 and 7.2.

- Precision:
 - Normal Traffic (0): 0.79 → When the model predicts normal, it is correct 79% of the time.
 - Malicious Traffic (1): 0.99 → When the model predicts malicious, it is correct 99% of the time. This high precision for malicious traffic means fewer false positives (normal samples misclassified as malicious).
- Recall (Sensitivity):
 - Normal Traffic: 1.00 → The model successfully detects all normal samples.
 - Malicious Traffic: 0.35 → The model only catches 35% of malicious samples. Many malicious samples are incorrectly classified as normal traffic, leading to a high rate of false negatives.
- F1-Score:
 - Normal Traffic: 0.88 → This score reflects a good balance between precision and recall for normal traffic.
 - Malicious Traffic: 0.51 → 0.51 (multi-class mode) and 0.52 (binary mode). These scores indicate challenges in effectively detecting malicious samples.
- Support:
 - There are 5879 normal samples and 2393 malicious samples in the test set.

```

Isolation Forest Accuracy: 0.8100
Isolation Forest Classification Report:

```

	precision	recall	f1-score	support
0	0.79	1.00	0.88	5879
1	0.99	0.35	0.51	2393
accuracy			0.81	8272
macro avg	0.89	0.67	0.70	8272
weighted avg	0.85	0.81	0.78	8272

Figure 7.1: Isolation Forest (IF) accuracy results for multi-class mode

```

=== Isolation Forest (Binary IT_B_Label) ===
Accuracy: 0.8108

```

	precision	recall	f1-score	support
0	0.79	1.00	0.88	5879
1	0.99	0.35	0.52	2393
accuracy			0.81	8272
macro avg	0.89	0.67	0.70	8272
weighted avg	0.85	0.81	0.78	8272

Figure 7.2: Isolation Forest (IF) accuracy results for binary mode

The confusion matrix, visualized in Figures 7.3 and 7.4, provides insight into the model's classification performance:

- True Negatives (TN): 5867
 - Normal samples correctly classified as normal.
- False Positives (FP): 7
 - Normal samples incorrectly classified as malicious, indicating good precision.
- False Negatives (FN): 1558
 - Malicious samples incorrectly classified as normal, indicating a recall issue for malicious class.
- True Positives (TP): 835
 - Malicious samples correctly identified as malicious.

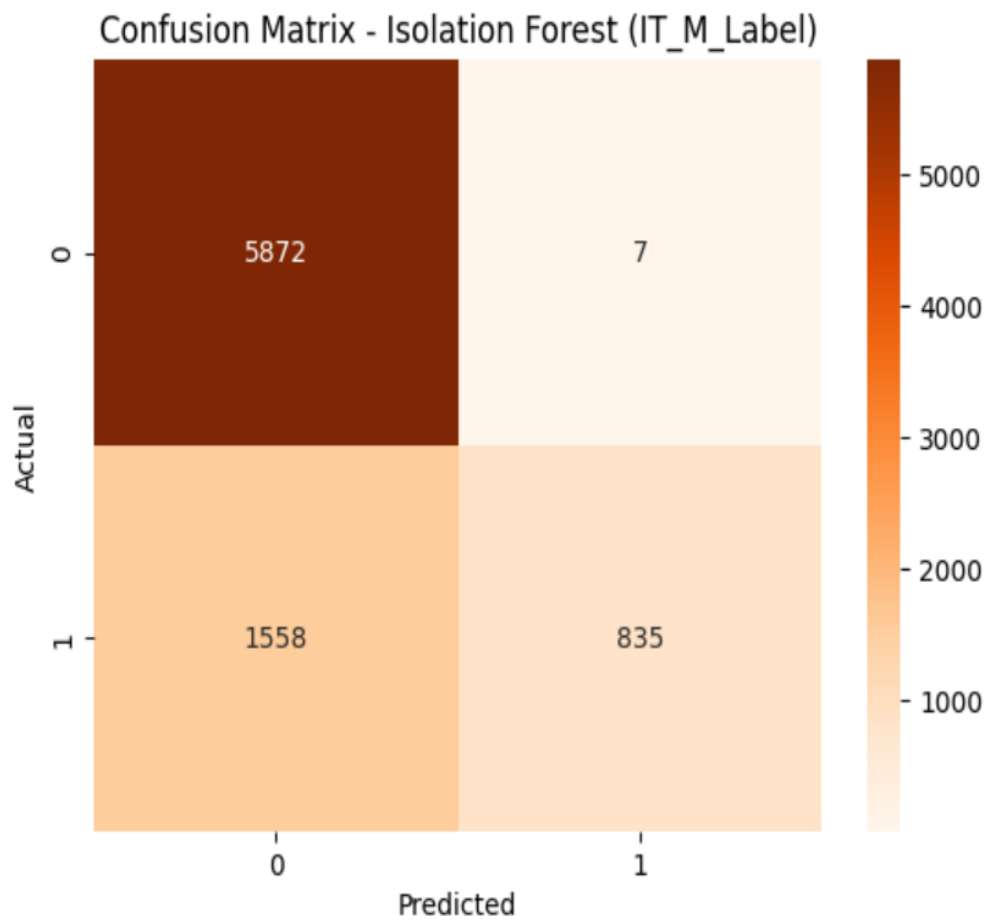


Figure 7.3: Isolation Forest Confusion Matrix for multi class mode

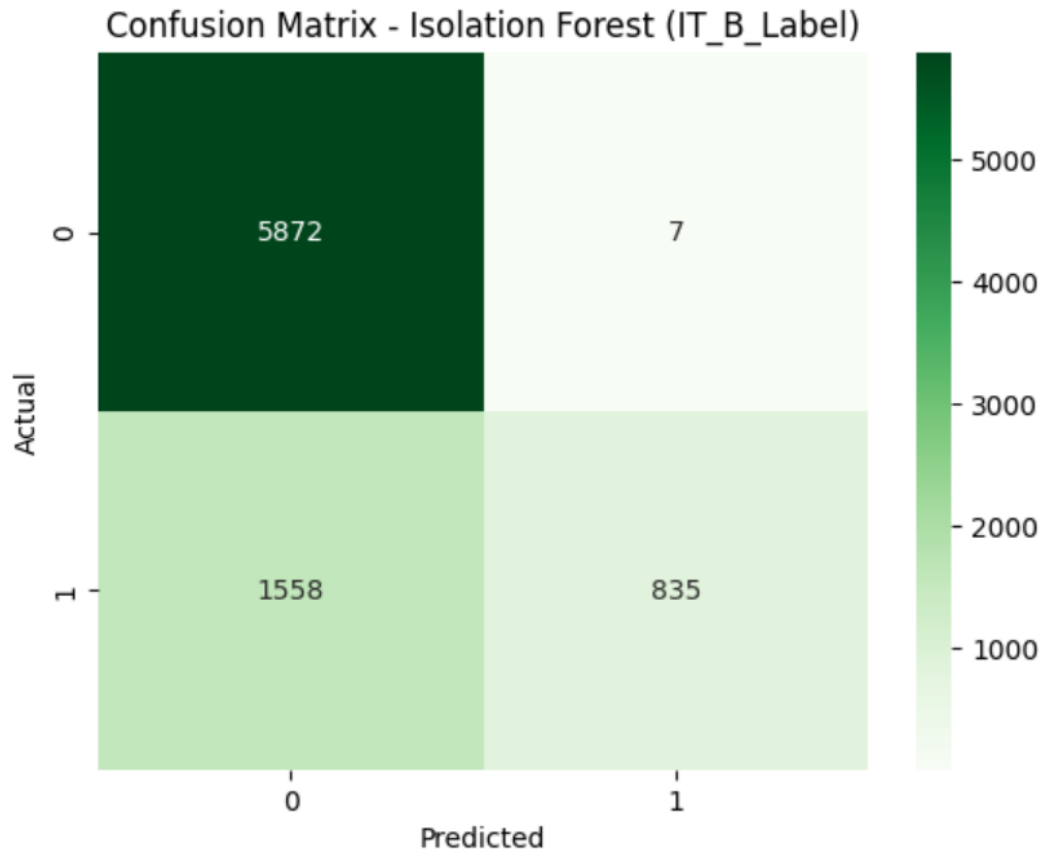


Figure 7.4: Isolation Forest Confusion Matrix for binary mode

7.4.2 Random Forest

The Random Forest model achieved an accuracy of 96.2% for multi class mode and 95.2% for binary mode, demonstrating its ability to correctly classify the majority of instances, as illustrated in Figures 7.5 and 7.6. Key observations for multi-class mode include:

- The model excels across all classes, especially for Class 0 (Normal).
- Class 2 (IP-Scan) has the lowest recall at 0.04, indicating that the model misses many instances of this class.
- The model maintains high precision (close to 1.00) across all classes, signifying that it generates very few false positive predictions.

```

=== Random Forest (Multi-class IT_M_Label) ===
Accuracy: 0.9626

```

	precision	recall	f1-score	support
0	0.96	0.99	0.98	5879
1	0.99	0.98	0.98	872
2	0.67	0.04	0.08	91
3	0.98	0.87	0.92	444
4	0.90	0.63	0.74	251
5	0.99	0.98	0.98	735
accuracy			0.96	8272
macro avg	0.91	0.75	0.78	8272
weighted avg	0.96	0.96	0.96	8272

Figure 7.5: Random Forest (RF) Accuracy Results for multi class mode

Key observations for binary mode include:

- The model is very good at detecting normal traffic and generally performs well in identifying attacks, though its effectiveness in detecting attacks is slightly lower.
- While the high precision and F1-score for attacks indicate reliability, a recall of 0.88 suggests that some attacks may be going undetected.

```

=== Random Forest (Binary IT_B_Label) ===
Accuracy: 0.9520

```

	precision	recall	f1-score	support
0	0.95	0.98	0.97	5879
1	0.95	0.88	0.91	2393
accuracy			0.95	8272
macro avg	0.95	0.93	0.94	8272
weighted avg	0.95	0.95	0.95	8272

Figure 7.6: Random Forest (RF) Accuracy Results for binary mode

The confusion matrix provides a detailed view of how the Random Forest classifier performed by comparing actual labels with predicted labels, as shown in figure 7.7. Each row corresponds to a specific type of network activity or cyberattack:

- **Class 0:** "Normal" traffic (legitimate, non-malicious network activity).
- **Class 1:** "DDoS" (Distributed Denial-of-Service) attack, where multiple systems flood a target with excessive traffic.
- **Class 2:** "IP-Scan" which involves scanning a network to identify active IP addresses.
- **Class 3:** "MITM" (Man-in-the-Middle), where an attacker intercepts and possibly alters communication between two parties.
- **Class 4:** "Port-Scan" an attack method used to probe systems for open ports.
- **Class 5:** "Replay Attack" where an attacker intercepts a valid data transmission and replays it to deceive the system.

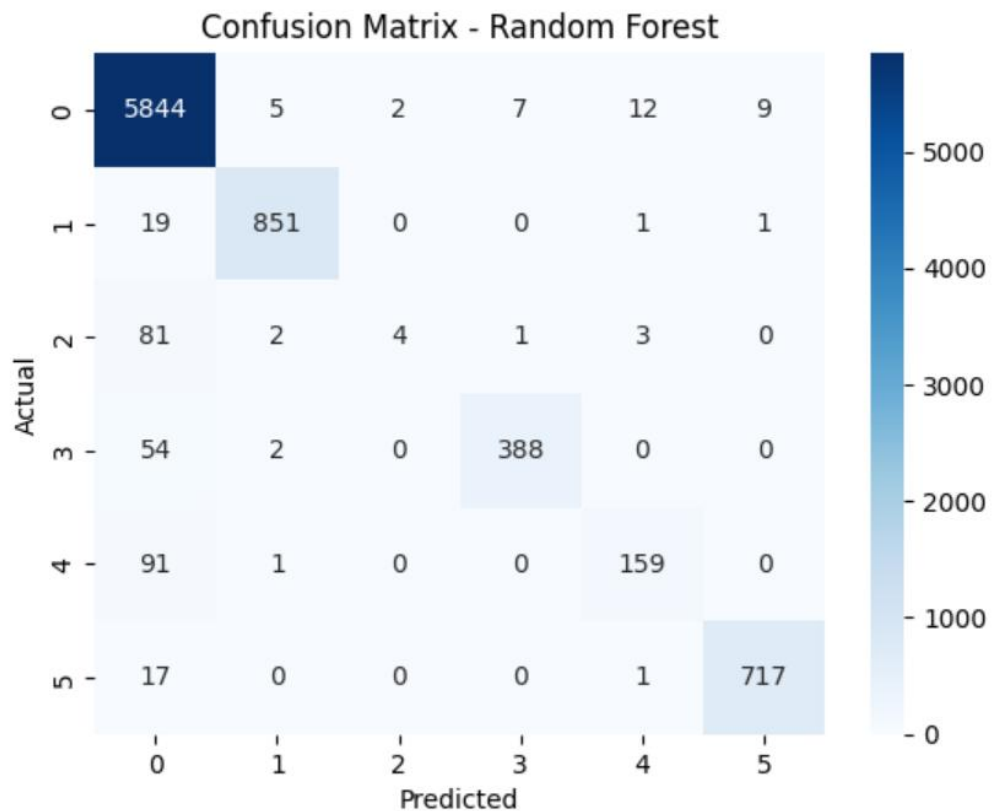


Figure 7.7: Random Forest Confusion Matrix for multi class mode

The diagonal values (from top-left to bottom-right) in the confusion matrix represent correct classifications for multi class mode, while off-diagonal values indicate misclassifications or errors.

The model demonstrates strong performance for the majority of classes, particularly Class 0 (Normal), Class 1 (DDoS), and Class 5 (Replay Attack).

Class 0 (Normal) is perfectly classified, with 5,844 out of 5,879 instances correctly predicted. Similarly, Class 1 (DDoS) achieves high precision and recall, with 851 out of 872 instances classified correctly. Class 5 (Replay Attack) is also well-identified, with 717 out of 735 instances classified without errors.

However, there is slight misclassification in Class 2 (IP-Scan). Out of 91 actual instances, the model correctly classified 4. This suggests that Class 2 and Class 4 have overlapping features, leading to classification confusion.

For Class 3, 388 instances were correctly classified, while 56 were misclassified. Similarly, Class 4 had 159 correct classifications, but 92 instances were misclassified, primarily as Class 0 or Class 1. These misclassifications indicate some feature similarities among these attack types, affecting the model's ability to distinguish them completely, as shown in Figure 7.7.

The confusion matrix as shown in Figure 7.8 represents the performance of the Random Forest classifier on a binary anomaly detection task using the IT_B_Label, where class 0 indicates normal traffic and class 1 represents anomalous or attack traffic. The matrix summarizes the number of correct and incorrect predictions made by the model, divided into four categories: true positives, false positives, true negatives, and false negatives.

From the matrix, we observe that the model correctly predicted 5,768 instances of normal traffic as normal (true negatives), and 2,107 instances of attack traffic as attack (true positives). These numbers indicate that the model is more successful in distinguishing between normal and anomalous traffic. However, there were 111 false

positives—cases where normal traffic was incorrectly classified as an attack—which may raise unnecessary alerts. More significantly, there were 286 false negatives, meaning actual attacks were mistakenly classified as normal traffic, which could potentially allow malicious activity to go undetected.

Overall, the confusion matrix confirms that the model performs well, with high accuracy. Nevertheless, the presence of false negatives highlights an area for improvement, as missing attacks can be critical in security contexts.

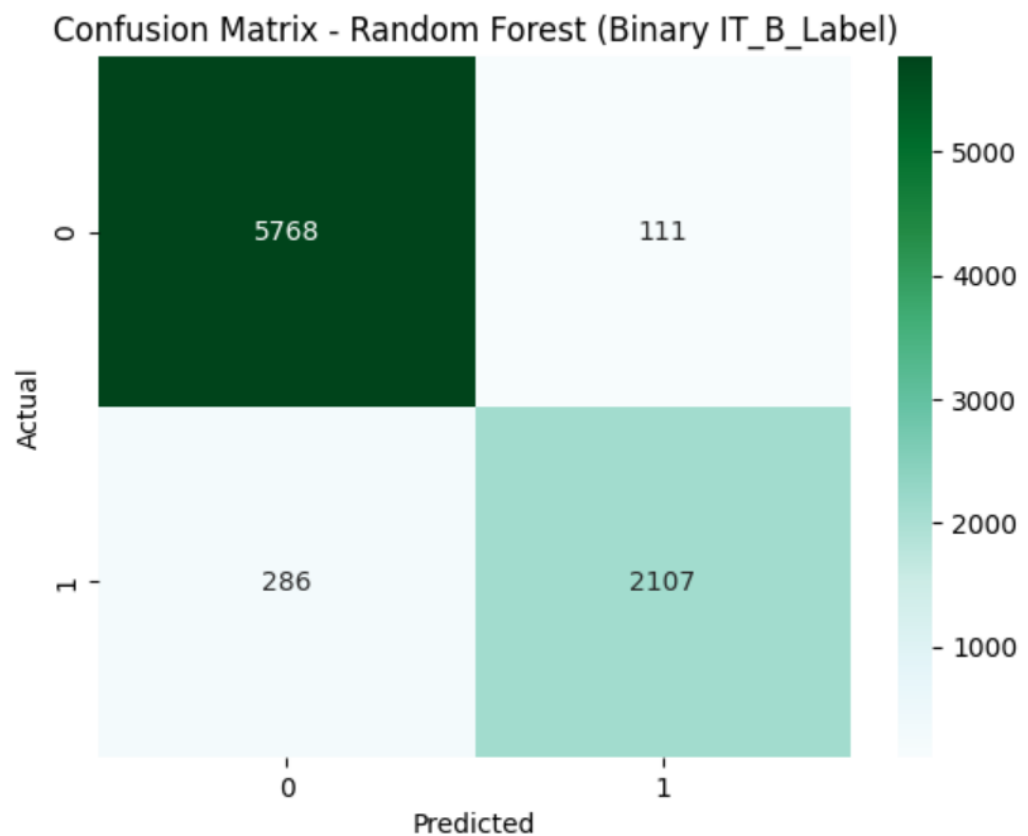


Figure 7.8: Random Forest Confusion Matrix for binary mode

7.4.3 SVM

The results presented in Figure 7.9 are from a SVM model applied in a multi-class classification setting using the IT_M_Label, which represents different types of traffic or attacks in the ICS dataset. The model achieved an overall accuracy of 85.86%, indicating a strong performance in correctly classifying samples across all classes.

However, a closer examination of precision, recall, and F1-scores for each class reveals some important performance variations.

For class 0, which represents normal traffic, the model performed very well with a recall of 1.00 and an F1-score of 0.91, indicating that almost all normal traffic was correctly classified. Similarly, class 3 also shows strong performance with high precision (0.99), recall (0.83), and F1-score (0.91). The strong performance in these classes can be attributed to the large number of samples for these classes, which makes them easier for the model to learn.

However, for less frequent attack classes such as 2, 4, and 5, the performance drops significantly. For example, class 2 has zero precision and recall, meaning the model failed to identify any instances of this class correctly. Class 4 has a recall of only 0.02, and class 5 has a recall of 0.50, showing that many of their instances were misclassified as other classes. These poor scores are likely due to class imbalance—the smaller number of samples in these classes makes it difficult for the model to learn their patterns accurately.

The macro average F1-score is 0.54, reflecting the overall fairness of the model across all classes by treating them equally. In contrast, the weighted average F1-score is 0.83, which takes into account the support (number of samples) in each class and is pulled up by the strong performance on the majority classes.

```

=== SVM (Multi-class IT_M_Label) ===
Accuracy: 0.8586

```

	precision	recall	f1-score	support
0	0.83	1.00	0.91	5879
1	0.99	0.56	0.71	872
2	0.00	0.00	0.00	91
3	0.99	0.83	0.91	444
4	1.00	0.02	0.03	251
5	1.00	0.50	0.66	735
accuracy			0.86	8272
macro avg	0.80	0.48	0.54	8272
weighted avg	0.87	0.86	0.83	8272

Figure 7.9: SVM for multi class mode

The binary classification results presented in Figure 7.10 for the SVM model using the IT_B_Label column demonstrate a moderate overall performance, with an accuracy of 86.16%. This means the model correctly classified 86.16% of the total samples in the dataset.

When we examine the class-wise performance, the model shows strong results for class 0 (normal traffic). It achieved a precision of 0.84, indicating that 84% of the samples predicted as normal were actually correct. More impressively, it achieved an excellent recall of 0.99, meaning it successfully identified nearly all actual normal traffic instances. This high recall contributes to a robust F1-score of 0.91, reflecting a good balance between precision and recall.

However, for class 1 (attack traffic), the model's performance for attack traffic is more limited. It achieved a very high precision of 0.98, suggesting that most predicted attacks were accurate. However, the recall dropped to 0.53, indicating that nearly half of the actual attacks went undetected. This lower recall significantly impacts the F1-score, which stands at 0.69, highlighting the model's challenges in fully capturing attack patterns.

Overall, the macro average F1-score is 0.80, suggesting a reasonable balance across both classes when treated equally. The weighted average F1-score is 0.85, slightly elevated due to the larger sample size in class 0, which the model handles more effectively.

While the SVM model effectively identifies normal traffic and demonstrates high precision in attack detection, the lower recall for attack instances raises concerns about undetected security threats. To enhance performance, the model could benefit from improvements such as class balancing, parameter tuning, or exploring alternative algorithms to better capture anomalies in the data.

```

=== SVM (Binary IT_B_Label) ===
Accuracy: 0.8616

```

	precision	recall	f1-score	support
0	0.84	0.99	0.91	5879
1	0.98	0.53	0.69	2393
accuracy			0.86	8272
macro avg	0.91	0.76	0.80	8272
weighted avg	0.88	0.86	0.85	8272

Figure 7.10: SVM for binary mode

The confusion matrix shown in Figure 7.11 serves as a visual report card for our SVM model's ability to categorize data points into six distinct classes, labeled 0 through 5. Each row in this table represents the actual, or true, class of a data point, while each column indicates the class that the model predicted for that same data point. The numbers populating the matrix reveal the counts of instances falling into each possible combination of actual and predicted classes, offering a granular view of the model's performance.

The diagonal elements, running from the top-left corner to the bottom-right, are particularly insightful as they represent the instances where the model's prediction perfectly aligned with the actual class. For instance, we see a substantial 5879 instances of class 0 were correctly identified. Similarly, 485 instances of class 1, 369 instances of class 3, and 365 instances of class 5 were correctly classified. However, the model shows a stark weakness with classes 2 and 4, achieving only 0 and 4 correct predictions, respectively. These diagonal values are often referred to as true positives for their corresponding classes.

Conversely, the off-diagonal elements illuminate the types and frequencies of errors the model makes. These numbers indicate instances where the predicted class differed from the actual class. For example, the '385' in the first column and second row tells us that 385 instances that were actually of class 1 were mistakenly predicted as class 0. These off-diagonal values represent false positives from the perspective of the predicted class and false negatives from the perspective of the actual class. Analyzing these off-

diagonal elements for each class provides critical information about the specific types of misclassifications the model tends to make.

Overall, this confusion matrix illustrates the SVM model's strong performance in identifying class 0 and reasonable accuracy for classes 1, 3, and 5. However, its significant difficulties with classes 2 and 4 indicate areas for improvement.

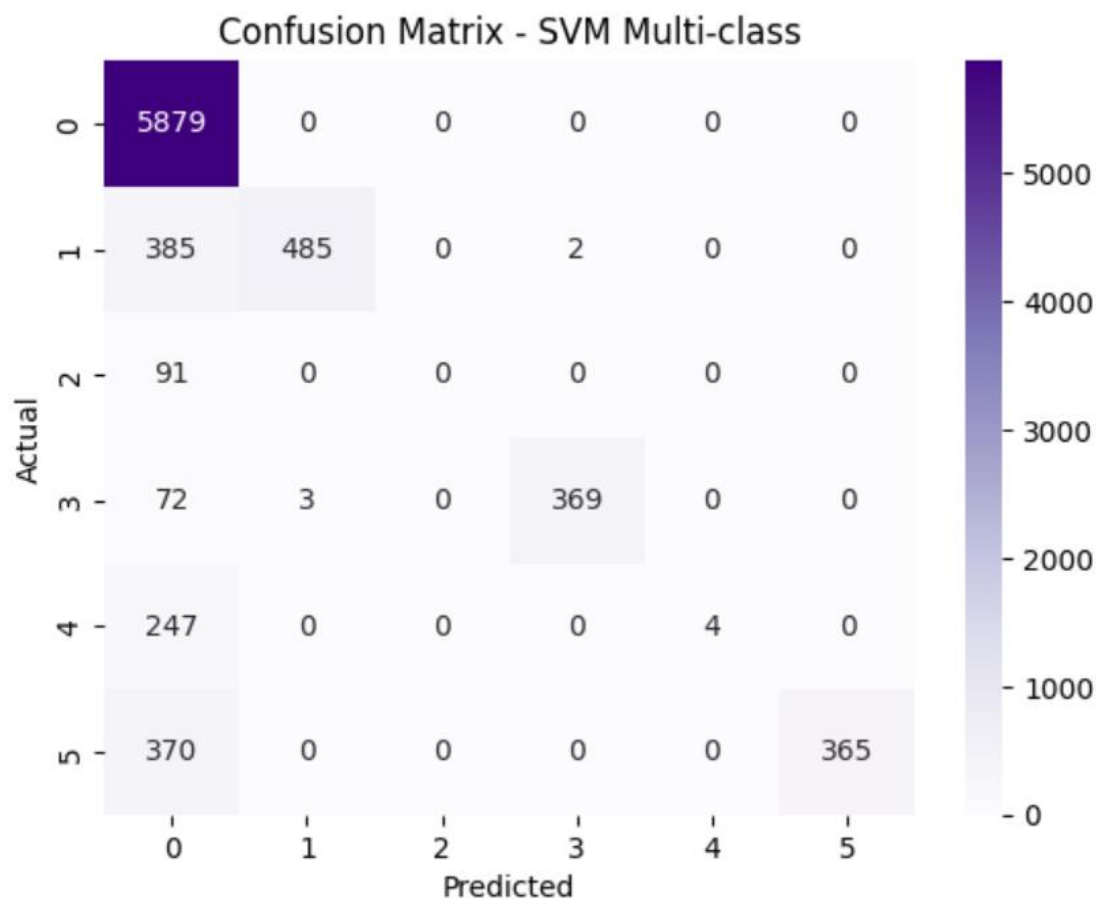


Figure 7.11: SVM Confusion Matrix for multi class mode

In the binary classification shown in Figure 7.12, our model is tasked with distinguishing between two classes, 0 and 1. This confusion matrix provides a clear visual summary of how well our SVM model performs this task by comparing the actual class of each data point with the class predicted by the model. The rows represent the true, or actual, classes, while the columns represent the classes predicted by the SVM.

The numbers within the matrix indicate the count of data points that fall into each of these four possible outcomes.

- The top-left cell contains 5848, represents the True Positives for class 0. This means there were 5848 instances were correctly classified as class 0 (normal).
- The top-right cell, with the value 31, represents the False Positives for class 0 (or False Negatives for class 1).
- The bottom-left cell, showing 1114, represents the False Negatives for class 0 (or False Positives for class 1).
- Finally, the bottom-right cell, containing 1279, indicating that these instances were correctly identified as class 1.

In essence, the diagonal elements (5848 and 1279) signify correct predictions, while the off-diagonal elements (31 and 1114) highlight the errors made by the model. The model effectively identifies instances of class 0; however, the higher number of false negatives for class 0 compared to false positives indicates potential areas for improvement. This detailed breakdown allows us to assess specific types of errors our binary SVM model is making, providing valuable insights for future enhancements.

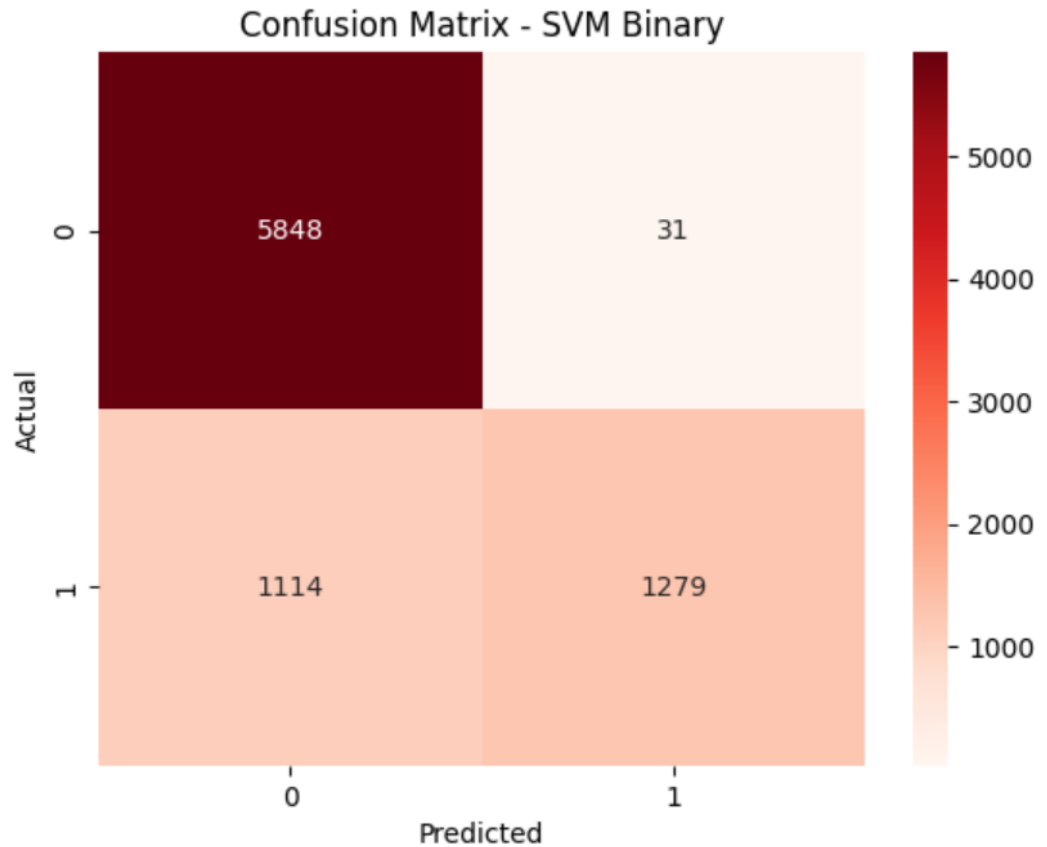


Figure 7.12: SVM Confusion Matrix for binary mode

7.4.4 Accuracy Comparison

The results summarized in Tables 7.1 and 7.2 highlight the performance of different models across multi-class and binary classification modes:

- Accuracy: In multi-class mode, Random Forest outperforms with an accuracy of 96.2%, followed by SVM at 85.8% and Isolation Forest at 81%. In binary mode, Random Forest again leads with an accuracy of 95.2%, while SVM achieves 86.1% and Isolation Forest remains at 81%.
- Macro Avg F1-score: In multi-class mode, Random Forest boasts a higher macro average F1-score of 0.91, compared to Isolation Forest at 0.89 and SVM at 0.80. For binary mode, Random Forest also excels with a macro average of 0.95, while SVM scores 0.91 and Isolation Forest is lower at 0.48.

- Weighted Avg F1-score: In multi-class mode, Random Forest maintains a strong performance with a weighted average F1-score of 0.96, while SVM achieves 0.87 and Isolation Forest is lower at 0.85. For binary mode, Random Forest again leads with a weighted average of 0.95, with SVM at 0.88 and Isolation Forest at 0.85.

Metric	Random Forest	Isolation Forest	SVM
Accuracy	96.2%	81%	85.8%
Macro Avg F1-score	0.91	0.89	0.80
Weighted Avg F1-score	0.96	0.85	0.87
Misclassification of Minority Classes	Very Low	Very High	Mid

Table 7.1: Comparison of Models for multi class mode

Metric	Random Forest	Isolation Forest	SVM
Accuracy	95.2%	81%	86.1%
Macro Avg F1-score	0.95	0.48	0.91
Weighted Avg F1-score	0.95	0.85	0.88
Misclassification of Minority Classes	Very Low	Very High	Mid

Table 7.2: Comparison of Models for Binary mode

7.5 Validity

7.5.1 Potential Threats to Validity

Potential threats to the validity of the anomaly detection model include overfitting, class imbalance, and dataset quality issues. Overfitting is particularly concerning with the Random Forest algorithm, as its high accuracy may indicate that it has learned patterns too specifically from the training data, which can reduce its ability to generalize to new data. Additionally, class imbalance in the dataset -where normal and malicious data are not evenly distributed- may negatively impact the performance of the Isolation Forest model. To address this issue, evaluation metrics such as the F1-score and confusion matrix analysis are used to provide a more balanced assessment of the model performance. Furthermore, the quality of the dataset is crucial for reliable evaluations; Any inconsistencies in the dataset labeling could lead to misleading results, affecting the model's effectiveness in accurately detecting anomalies.

7.6 Summary of Findings (Discussion)

7.6.1 Explanation of Outcomes

The outcomes of the ICS anomaly detection project highlight the performance of three models: Random Forest, Isolation Forest, and Support Vector Machine (SVM), applied using both IT_B_Label and IT_M_Label in binary and multi-class classification settings. Each model was evaluated based on its ability to detect and classify anomalies in ICS network traffic.

The Random Forest model performed the best in classifying traffic into multiple categories, including normal activity and various types of attacks. Its performance was measured through accuracy scores, classification reports, and confusion matrices. High accuracy values suggest that the model effectively distinguishes between different types of ICS anomalies. Confusion matrices were particularly useful for identifying misclassified samples, guiding improvements in feature selection or hyperparameter tuning.

In contrast, the Isolation Forest model was performing better in binary classification, identifying traffic as either normal or anomalous. This unsupervised technique is well-suited for general anomaly detection, especially in scenarios where the specific type of attack is unknown or unlabeled. Evaluation metrics, including false positives and false negatives, provided insights into the model's reliability. A low accuracy rate suggested the need to fine-tune parameters such as the contamination rate or to explore alternative anomaly detection approaches.

The SVM model was also implemented in both binary and multi-class conditions, SVM demonstrated varying effectiveness depending on the data distribution and feature space. In binary mode, it showed potential in identifying anomalous behavior, while in multi-class settings, its performance depended heavily on data separability and balance among classes.

Overall, the comparison across models and labeling strategies demonstrated that each approach has unique strengths. Random Forest performed well for detailed anomaly classification, Isolation Forest offered a lightweight and effective binary anomaly detector, and SVM provided a flexible option that could adapt to both binary and multi-class needs. The final choice of model and labeling approach depends on the specific security requirements of the ICS environment—whether broad anomaly detection or precise attack classification is prioritized.

7.6.2 Implications of Results

The results provide valuable insights into the effectiveness and practical application of different machine learning techniques in industrial control environments. The performance variation observed across Random Forest, Isolation Forest, and SVM models—each tested on both IT_B_Label and IT_M_Label using binary and multi-class classification—offers several important implications for the design and deployment of ICS security systems.

First, the high accuracy and detailed classification capabilities of the Random Forest model in multi-class settings suggest that it is well-suited for environments where understanding the specific type of anomaly or attack is critical. This supports its use in advanced ICS monitoring systems that aim to categorize threats and apply targeted mitigation strategies. However, the model's sensitivity to class imbalance also implies a need for careful preprocessing, such as data augmentation or class weighting, to ensure reliable performance in real-world settings.

The results from the Isolation Forest model highlight its utility as a general anomaly detector, especially in situations where labeled data is limited or unavailable. Despite its lower performance in some cases, its unsupervised nature and simplicity make it a practical choice for early-stage or lightweight anomaly detection systems. The findings also suggest that tuning parameters such as contamination can significantly influence its effectiveness, pointing to the need for context-specific calibration.

SVM's performance across binary and multi-class classifications illustrates its flexibility and robustness under certain conditions. While it may not always outperform other models, its capacity to handle complex decision boundaries and its solid

performance in binary classification scenarios imply that it could serve as a reliable component in hybrid detection systems. These results emphasize the importance of selecting algorithms not only based on accuracy but also on the nature of the data, computational constraints, and operational goals.

In summary, the project underscores that there is no one-size-fits-all solution for ICS anomaly detection. Instead, selecting the most appropriate model depends on the specific detection objectives—whether broad anomaly identification or detailed classification of threat types—as well as the quality and structure of the available data. These insights can guide future research and real-world implementations in designing more resilient and adaptive ICS cybersecurity systems.

7.6.3 Solution Review

The proposed solution effectively fulfills the core objectives of the project by leveraging machine learning models to detect anomalies and classify attack types within ICS network traffic. By implementing and evaluating Random Forest, Isolation Forest, and SVM across both binary and multi-class classification settings using IT_B_Label and IT_M_Label, the project demonstrates a comprehensive approach to addressing ICS cybersecurity challenges.

The Random Forest model consistently delivered strong performance in multi-class classification, accurately distinguishing between various types of attacks. This reinforces its suitability for detailed anomaly detection systems that require precise identification of threats. The SVM model also showed promising results, particularly in binary classification tasks, indicating its potential as a lightweight yet effective anomaly detector when tuned appropriately.

However, the evaluation also revealed a significant disparity in performance, particularly in the Isolation Forest model's ability to recall minority class anomalies. While Isolation Forest is valuable for unsupervised anomaly detection, its limited sensitivity to certain types of attacks suggests that further optimization is necessary. Enhancing its recall could involve adjusting the contamination parameter, refining feature selection, or incorporating ensemble strategies to improve detection accuracy for rare or subtle anomalies.

In addition, the results highlight the importance of data preprocessing techniques—such as class balancing, scaling, and feature engineering—in maximizing model performance. These steps play a crucial role in ensuring that models are not only accurate but also generalizable across varying ICS scenarios.

Overall, the solution provides a strong foundation for ICS anomaly detection, with clear paths for future refinement. Integrating model tuning, hybrid approaches, or even real-time data adaptation could further enhance the system's robustness and effectiveness in real-world industrial environments.

7.6.4 Project Review

Overall, the project achieved its primary objectives of anomaly detection and attack classification. Future work could explore hybrid approaches or integrate advanced techniques, such as deep learning, to further improve performance.

7.6.5 Key Skills

During this project, key skills developed include:

- **Machine Learning Implementation:** Gained hands-on experience with classification models in Python.
- **Data Preprocessing:** Enhanced understanding of data cleaning, encoding, and scaling techniques.
- **Model Evaluation:** Learned to critically assess model performance using appropriate metrics and visualization tools.

7.6.6 Summary of Discussion

The evaluation phase demonstrated that the proposed anomaly detection system is both effective and reliable in a CAN-based ICS environment. While the Random Forest model showed exceptional performance, the Isolation Forest's results identified areas for potential enhancement. Insights gained through this testing process will inform

future improvements and the potential for real-world deployment of the anomaly detection system.

Chapter 8: Conclusion and Future Works

This chapter summarizes the key findings of the project and its contributions to the field of anomaly detection in CAN-based Industrial Control Systems (ICS). The project aimed to develop an efficient detection mechanism utilizing machine learning techniques—specifically Random Forest, Isolation Forest, and SVM—to identify anomalies within ICS traffic data. The project successfully demonstrated the feasibility of these models in detecting malicious activities while highlighting their strengths and limitations. This chapter provides an overall analysis of the study, discusses its contributions, and suggests future enhancements to improve the system.

8.1 Conclusion

The increasing reliance on Industrial Control Systems (ICS) in critical sectors such as manufacturing, energy, and transportation underscores the importance of robust cybersecurity measures. Our project addresses the vulnerabilities inherent in systems utilizing the Controller Area Network (CAN) protocol, which, while effective for real-time communication, is susceptible to various cyber threats, including DoS, spoofing, fuzzy and data injection attacks.

Through the development of a machine learning-based anomaly detection system, we demonstrate the potential to identify unusual behaviors within CAN-based ICS networks. By utilizing the ICS-Flow dataset, we implemented both supervised and unsupervised learning techniques, allowing for a flexible approach to detect known and unknown threats. The evaluation of different machine learning models provided insights into their performance and highlighted the effectiveness of our approach. Our comprehensive UML diagrams and structured methodology ensure that the system is not only functional but also adaptable for future enhancements.

The implementation of Random Forest, Isolation Forest and SVM for both binary classification and multi-class classification has yielded promising results. These findings contribute to the growing body of knowledge on cybersecurity in ICS environments and validate the project's hypothesis that supervised and unsupervised learning techniques can enhance anomaly detection performance in CAN-based ICS.

8.2 Limitations

While the results of this study provide valuable insights into anomaly detection in Industrial Control Systems (ICS), several limitations must be acknowledged for a balanced interpretation of the findings:

- **Overfitting:** A key limitation is overfitting, particularly with the Random Forest model. Although the model demonstrated high accuracy in multi-class classification, this may indicate that it has learned patterns too specifically from the training data. Future work should explore regularization techniques or feature selection strategies to mitigate this issue.
- **Class Imbalance:** Class imbalance significantly affects the performance of the Isolation Forest model. Since anomaly detection datasets often contain disproportionately fewer malicious samples than normal ones, the model may struggle to detect minority attack types. The underperformance of the Isolation Forest in such cases suggests that techniques like data resampling, cost-sensitive learning, or synthetic data generation should be explored to enhance its ability to identify rare anomalies accurately.
- **Dataset Quality:** the quality of the dataset plays a crucial role in model effectiveness. Inconsistencies in labeling, missing values, or biased data distributions could lead to misleading evaluations and unreliable results. Ensuring high-quality, well-labeled datasets is essential for improving the reliability and fairness of anomaly detection models.
- **Evaluation Metrics:** While evaluation metrics such as F1-score, confusion matrix analysis, and precision-recall trade-offs have been employed to assess model performance comprehensively, future research should focus on developing more advanced strategies to handle dataset limitations and improve detection accuracy in ICS environments.

8.3 Future Works

To further enhance the effectiveness of the proposed anomaly detection system, the following future directions are suggested:

- **Dataset Expansion:** Incorporating more diverse and real-world ICS datasets to improve model generalization.
- **Model Optimization:** Exploring advanced machine learning techniques, such as deep learning, to improve accuracy and reduce false positives.
- **Real-Time Implementation:** Integrating detection models with ICS environments for real-time monitoring and response.
- **Feature Engineering Improvements:** Implementing automated feature selection methods to refine the feature set used for classification.

8.4 Summary

This project has successfully demonstrated the application of machine learning for anomaly detection in CAN-based ICS. Through the implementation of Random Forest, Isolation Forest, and SVM, the study highlights the potential of machine learning to enhance ICS security. While the models exhibited strong performance in detecting anomalies, future research should focus on optimizing model performance, expanding dataset diversity, and addressing the identified limitations. These enhancements will further strengthen cybersecurity measures in industrial control systems, mitigating risks associated with cyber threats.

References

- [1] Javier Martínez Torres, Carla Iglesias Comesaña , Paulino J.García-Nieto , “Review: machine learning techniques applied to cybersecurity”,published at Springer-Verlag GmbH Germany, part of Springer Nature 2019.
- [2] Mead Rashed Albediwi , Kishwar Sadaf , “A Framework for Cybersecurity Awareness in Saudi Arabia.”, published at Journal of Engineering and Applied Sciences, November 2023.
- [3] Chao Wang, Bailing Wang , Hongri Liu , Haikuo Qu , “Anomaly Detection for Industrial Control System Based on Autoencoder Neural Network”, published at Wireless Communications and Mobile Computing ,2020.
- [4] Tanmoy Kanti Das , Sridhar Adepu , Jianying Zhou , “Anomaly detection in Industrial Control Systems using Logical Analysis of Data” , published at Computers & Security , 2020.
- [5] Bertino, E., & Islam, N. “Botnets and Internet of Things security”. *Computer*, 50(2), 76-79,<https://doi.org/10.1109/MC.2017.62>,2017.
- [6] Sohrab Mokhtari ,Alireza Abbaspour , Kang K. Yen and Arman Sargolzaei , “A Machine Learning Approach for Anomaly Detection in Industrial Control Systems Based on Measurement Data”, published at Electronics ,10, 407. <https://doi.org/10.3390/electronics10040407>,2021.
- [7] Carnegie Mellon University“ *CERT Division reports on cybersecurity vulnerabilities: Industrial Control Systems (ICS)*,” published at Software Engineering Institute. Retrieved from <https://www.sei.cmu.edu/about/divisions/cert/>,2021.
- [8] Emre Koçyiğit, Mehmet Korkmaz, Ozgur Koray Sahingoz, Banu Diri. “Real-Time Content-Based Cyber Threat Detection with Machine Learning,” published at *Intelligent Systems Design and Applications* (pp. 1394-1403), 2021.

- [9] Aviraj Sinha , Michael Taylor, Nathan Srirama, Theodore Manikas, Eric C. Larson, and Mitchell A. Thornton , “Industrial Control System Anomaly Detection Using Convolutional Neural Network Consensus” , published at smu.edu , 2021.
- [10] Motaz Abdulaziz Almedires, Mohammed AlMaiah, and Dr. Mohammed Almaiah. “Cybersecurity in Industrial Control System (ICS).” Published at *International Conference on Information Technology (ICIT)*.,2021.
- [11] Anastasis Keliris , Charalambos Konstantinou,Nektarios Tsoutsos,Raghad Baiad,Michail Maniatakos , “Enabling multi-layer cyber-security assessment of Industrial Control Systems through Hardware-In-The-Loop testbeds” , published at 21st Asia and South Pacific Design Automation Conference (ASP-DAC) , 2016.
- [12] Nilufer Tuptuk, Stephen Hailes. “Identifying vulnerabilities of industrial control systems using evolutionary multiobjective optimization , published at *Computers and Security*, 2024.
- [13] Abdullah M. Alnajim , Shabana Habib , Muhammad Islam , Su Myat Thwin and Faisal Alotaibi , “A Comprehensive Survey of Cybersecurity Threats, Attacks, and Effective Countermeasures in Industrial Internet of Things” , published at <https://www.mdpi.com/journal/technologies> , 2023.
- [14] Christos Xenofontos, "Security Attacks in Programmable Logic Controllers (PLCs) in Industrial Control Systems (ICS)", Submitted to the University of Cyprus the degree of Bachelor of Science in Electrical Engineering, p.11, 2021.
- [15] M. Santhosh Kumar, C. R. Balamurugan, “A detailed review on self propelled safety monitoring system using can protocol”, published at World Journal of Modelling and Simulation · January 2017.
- [16] Omar El Idrissi, Abdellatif MEZRIOUI , Abdelhamid Belmekki, “Cyber Security challenges and Issues of Industrial Control Systems – Some Security Recommendations” , published at IEEE , 2020.
- [17] Nicholson, A., Webber, S., Dyer, S., Patel, T., & Janicke, H. “SCADA security in the light of cyber-warfare.” *Computers & Security*, 31(4), 418-436. <https://doi.org/10.1016/j.cose.2012.02.009>, 2012.

- [18] Wang, W., & Lu, Z. “Cyber security in the Smart Grid: Survey and challenges.” *Computer Networks*, 57(5), 1344-1371. <https://doi.org/10.1016/j.comnet.2012.12.017>, 2013.
- [19] Steven Hurd; Rhett Smith; Garrett Leischner , “Tutorial: Security in Electric Utility Control Systems” , published at IEEE , 2008.
- [20] Vassilios Vassilakis, Ioannis D. Moscholio, John Vardakas, “On the Digital Certificate Management in Advanced Metering Infrastructure Networks” , published at <https://www.researchgate.net/publication/318274768>, 2017.
- [21] Allumiax, “SCADA and its application in electrical power systems” published at Allumiax. <https://www.allumiax.com/blog/scada-and-its-application-in-electrical-power-systems,2024>.
- [22] City of Albemarle , “Advanced Meter Infrastructure. City of Albemarle” published at <https://www.albemarlenc.gov/departments/advanced-meter-infrastructure> , 2024.
- [23] SaudiGulf Projects, “*Saudi Arabia announces discovery of major oil and natural gas fields*”. Published at SaudiGulf Projects. <https://www.saudigulfprojects.com/2024/07/saudi-arabia-announces-discovery-of-major-oil-and-natural-gas-fields/> , 2024.
- [24] Clement Rog , “Our 101 guide to Agile project management ” , published at <https://slite.com/learn/agile>, 2021.
- [25] Andrei-Grigore Mari, Daniel Zinca and Virgil Dobrota , “Development of a Machine-Learning Intrusion Detection System and Testing of Its Performance Using a Generative Adversarial Network” , published at *Sensors* 2023.
- [26] Alireza Dehlaghi , “ICS-Flow” , <https://www.kaggle.com/datasets/alirezadehlaghi/icssim>, published at 2022
- [27] Rifkie Primartha, Bayu Adhi Tama, “Anomaly Detection using Random Forest: A Performance Revisited”, published at IEEE , 2017.
- [28] Md Saif Mahmud , Md Ashikul Islam , Md Maruf Rahman , Debashon Chakraborty, “Enhancing Industrial Control System Security: An Isolation Forest-

based Anomaly Detection Model for Mitigating Cyber Threats” , published at Journal of Engineering Research and Reports , 2024.

[29] Alireza Dehlaghi-Ghadim,Mahshid Helali Moghadamy, Ali Baladorz Hans Hansson , “Anomaly Detection Dataset for Industrial Control Systems” , published at Research Institute of Sweden (RISE) , 2023.

[30] Esra Altulaihan ,Mohammed Amin Almaiah and Ahmed Aljughaiman , “Anomaly Detection IDS for Detecting DoS Attacks in IoT Networks Based on Machine Learning Algorithms” , published at Sensors 2024 , <https://doi.org/10.3390/s24020713>.

APPENDICES

Appendix A: Project Plan

Final Year Student Graduation Project

Academic Year 2024-25/1445-46 (Semester II)

Industrial Control System (ICS) Anomaly Detection

1	Rand Mohammed	
2	Lama Khalid	
3	Ghada Abdullah	

Supervised by

Dr. Amani Mohammad

Project Concept and Plan

Abstract

With the widespread digitalization of our lives and the growing connection of devices to the internet, cybersecurity has become a leading research focus in computer engineering. The nature of the internet has certainly led to an increase in both the frequency of cyberattacks and the diversity of their methods.

Industrial control systems (ICS) are essential to industries such as manufacturing, energy, and transportation, forming the backbone of these sectors. However, as they integrate into more connected networks, they face significant cybersecurity risks. Today, the Controller Area Network (CAN) in ICS works reasonably well to provide robust and real-time communication between devices; however, it is not inherently secure and is vulnerable to many attacks, such as Denial of Service (DoS), spoofing, and data injection attacks. This project aims to mitigate these risks by developing an anomaly detection system for CAN-based ICS networks.

Our project leverages machine learning techniques to enhance ICS cyber-physical security. Specifically, we utilize both supervised and unsupervised learning approaches to detect anomalies in CAN traffic.

For supervised learning, we employ the Random Forest and SVM algorithms, both of which are particularly effective for anomaly detection. Random Forest excels in classification tasks by constructing multiple decision trees and aggregating their results for high accuracy. This technique helps in identifying known attack patterns within the dataset.

For unsupervised learning, we implement the Isolation Forest. It isolates outliers by constructing random partitions, making it well-suited for detecting previously unseen or novel cyber threats in ICS networks.

By applying these machine learning models to a dataset containing normal and malicious CAN traffic, our system aims to accurately identify attacks such as DoS, spoofing, fuzzy, replay, and data injection attacks. This proactive approach enhances the security of ICS environments, providing a robust defense against cyber threats.

Milestones & Deliverables

The following Table 9.1 shows the main project management points and verification of achievement for project objectives. Each deliverable indicates the completion of a specific project phase.

Milestone No	Milestone	Deliverable Name	Due Date	Delivered By	Status (completed -In Pogress -Not Started)
N1	Project Concept and Plan	Project Concept and Plan	5/2/2025	Project Leader	completed
N2	Thesis Report: Project Implementation	Chapter-6	18/2/2025	Project Leader	completed
N3	Thesis Report: Project Testing	Chapter-7	5/3/2025	Project Leader	completed
N4	Thesis Report: Conclusions and Future Work	Chapter-8	15/4/2025	Project Leader	completed
N5	Report Submission	Report	19/4/2025	Project Leader	completed
N6	Thesis Report: Final Thesis	Final Project document	23/4/2025	Project Leader	completed

Table 9.1: Milestones& Deliverables

Gantt chart

The Gantt chart presented in Figure 9.1 illustrates the project's timeline and key milestones. This chart helps track progress and ensures that all tasks are completed on schedule.

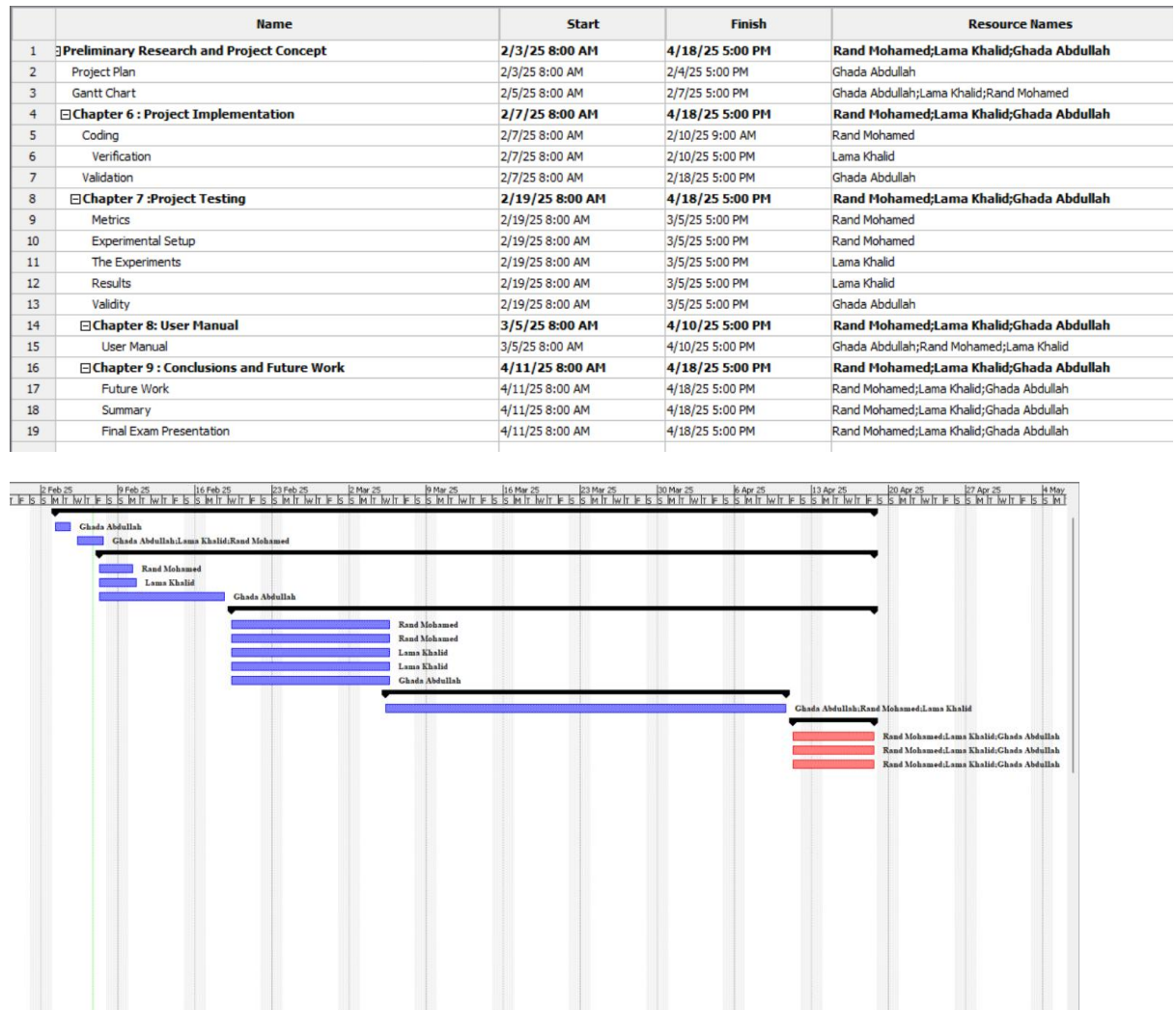


Figure 9.1: Gantt Chart

Appendix B: User Requirements Specification

1. Functional Requirements

ID	Requirement Description	Priority
FR1	The system shall ingest ICS network traffic data in CSV or similar structured formats.	High
FR2	The system shall preprocess the dataset, including feature scaling and label encoding.	High
FR3	The system shall provide performance evaluation using accuracy, confusion matrix, and classification report.	High
FR4	The system shall log results and store model performance for comparison.	Medium

Table 9.2: Functional Requirements

2. Non-Functional Requirements

ID	Requirement Description	Priority
NFR1	The system shall process datasets efficiently and complete training within a reasonable timeframe (< 5 minutes per model).	High
NFR2	The system shall be implemented using Python and be executable in environments such as Google Colab.	High
NFR3	The system shall be modular, with clear separation between data preprocessing, model training, and evaluation.	High
NFR4	The system shall be easily extendable to include additional machine learning algorithms in the future.	Medium
NFR5	The system should produce readable and interpretable outputs for use in academic reporting and research documentation.	High

Table 9.3: Non-Functional Requirements

Appendix C: Software Testing Document

1. Introduction

This document outlines the software testing conducted to verify the functionality, reliability, and performance of the ICS anomaly detection system. The goal of testing is to ensure that each component of the machine learning pipeline—from data preprocessing to model evaluation—operates as intended.

2. Testing Strategy

The testing process was divided into the following levels:

- Unit Testing: Individual functions (e.g., data loading, preprocessing, prediction) were tested in isolation.
- Integration Testing: Interactions between components (e.g., model training and evaluation) were tested.
- Performance Testing: Time complexity and efficiency were monitored during model training and testing.

3. Testing Environment

- Platform: Google Colab / Jupyter Notebook
- Language: Python 3.x
- Libraries: sklearn, numpy, pandas, matplotlib, seaborn
- Dataset: ICS-Flow dataset

4. Testing Cases

ID Test case	Test Description	input	Expected Output	Status
TC01	Load dataset successfully	CSV File	DataFrame with correct shape and columns	Pass
TC02	Normalize features using StandardScaler	Feature matrix	Transformed feature matrix (mean=0, std=1)	Pass
TC03	Train Random Forest, Isolation Forest and SVM (binary and multi-class)	Pre-processed data	Trained model without error	Pass

TC04	Perform prediction	Test data	Predictions as class labels	Pass
TC05	Evaluate using classification report	True vs predicted labels	Accuracy, Precision, Recall, F1-score	Pass
TC06	Visualize confusion matrix	y_true, y_pred	Displayed plot	Pass

Table 9.4: Test Cases Table

5. Bugs Reports

ID Bug	Description	Resolution
Bug01	Isolation Forest returned low accuracy on multi-class labels	Resolved by transforming labels to binary

Table 9.5: Bugs Table

Appendix D: Software Validation Document

1. Introduction

This validation document ensures that the implemented software (Python script for anomaly detection in ICS data) works as expected in terms of loading, processing, and evaluating machine learning models for anomaly detection.

- Scope: This software covers:
 - Data loading and preprocessing (e.g., handling missing values, label encoding).
 - Model training and prediction using Random Forest, Isolation Forest, and Support Vector Machine (SVM).
 - Model evaluation using accuracy, classification report, and confusion matrix.

2. Validation Objectives

- Ensure that the software loads and preprocesses the dataset correctly.
- Verify that the models (Random Forest, Isolation Forest, SVM) are trained and evaluated accurately.
- Validate that the performance metrics (accuracy, classification report, confusion matrix) are computed correctly.

3. Test Execution

The above test cases will be executed sequentially, and their results will be documented accordingly.

4. Validation Results

The software passed all test cases with the following results:

- Data Preprocessing: No errors in encoding or handling missing values.
- Model Evaluation: The performance of the models across multi-class and binary classification modes are shown in Tables 9.6 and 9.7.

Metric	Random Forest	Isolation Forest	SVM
Accuracy	96.2%	81%	85.8%
Macro Avg F1-score	0.91	0.89	0.80
Weighted Avg F1-score	0.96	0.85	0.87
Misclassification of Minority Classes	Very Low	Very High	Mid

Table 9.6: Model Evaluation for multi class mode

Metric	Random Forest	Isolation Forest	SVM
Accuracy	95.2%	81%	86.1%
Macro Avg F1-score	0.95	0.48	0.91
Weighted Avg F1-score	0.95	0.85	0.88
Misclassification of Minority Classes	Very Low	Very High	Mid

Table 9.7: Model Evaluation for Binary mode

Appendix E: Experimental Data and Results Document

1. Dataset Description

The experiments were conducted using ICS network traffic data containing features extracted from real-time industrial protocols. The dataset includes both benign and malicious traffic samples, labeled under two schemes:

- IT_B_Label: Represents binary classification (0 = normal, 1 = anomaly).
- IT_M_Label: Represents multi-class classification (0 = normal, 1–n = various attack types).

2. Preprocessing Steps

- Feature Selection: Irrelevant features were removed based on domain knowledge and correlation analysis.
- Label Transformation: For binary classification, all non-zero labels were converted to 1 (indicating an attack).
- Normalization: Features were standardized using StandardScaler to improve model convergence.
- Train-Test Split: The dataset was divided into training (80%) and testing (20%) subsets using stratified sampling.

3. Models Used

Model	Type	Classification Mode
Random Forest	Supervised	Multi-class & Binary
Isolation Forest	Unsupervised	Multi-class & Binary
SVM	Supervised	Multi-class & Binary

Table 9.8: Models Used

4. Evaluation Metrics

- Accuracy
- Precision, Recall, F1-score (via classification report)
- Confusion Matrix